Improving Structural Estimation with Sequential Neural Posterior Estimation

By Cameron Fen

Motivation

- Solving vs Estimation

- Solving heterogeneous agent (HA) models, widely becoming the go to approach, requires handling infinite dimensional objects (probability distributions)

  - Pain to estimate because it’s a pain to solve

  - Also, estimation must also handle these probability distributions

Current Problems in Bayesian Estimation

- Likewise, Bayesian estimation techniques are problematic:
  - Workhorse Metropolis-Hastings Markov Chain Monte Carlo (MCMC) has difficulty converging in both HA and non-HA models
  - MCMC has difficulty with under-identified and multi-modal posteriors
- MCMC has difficulties with estimating repersentitive agent models
  - Moving from representative agent to HA → more estimation problems

Sequential Neural Posterior Estimation (SNPE): A Solution to Estimation Problems

- As a simulation-based method, SNPE combines the best of both worlds:
  - No likelihood function required, like MSM, but always efficient when model well specified
  - Can Bayesian estimate, but greater speed/accuracy vs likelihood-based MCMC
    - Theoretical reasons why SNPE is more accurate with multi-modal posteriors
  - Estimation is independent of and not affected by the model or solution method
- This is applied computational research so I'm going to walk you through using my method to estimate HA/non-HA models, few if any proofs

Simulation-based Estimation Literature

Bayesian Background

Bayes Rule

$$P(\theta|X) = \frac{P(X|\theta)P(\theta)}{P(X)}$$

- Example: Given equation: $X = \theta + \sigma\epsilon$
- ...the likelihood is Z-score of $\frac{X'-\theta}{\sigma}$ where X' is the real data
- However, simulating from the likelihood involves simulating X, conditional on some fixed θ and σ, by drawing from ϵ and building a surrogate function to evaluate X' likelihood

Variable	Discription
Θ	Parameters (Discount Rate, Capital Share etc)
X	Data (GDP, Consumption etc.)
P(Θ)	Prior
P(X|Θ)	Likelihood
P(X)	Normalizing Term
P(Θ|X)	Posterior (the Obejctive)

Sampling

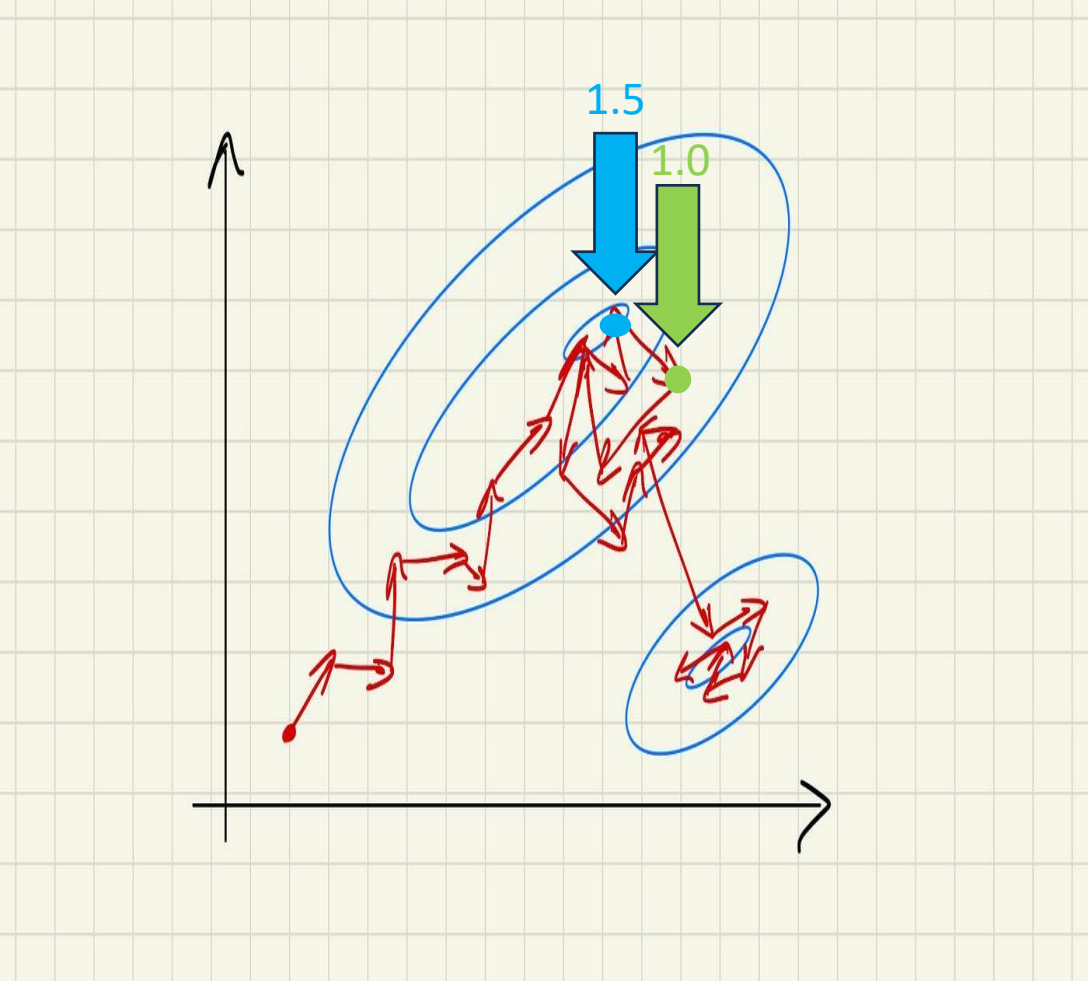
Overview of How MCMC Works

- MCMC samples from the posterior according to the likelihood ratio of points in the space

- The likelihood MCMC moves from a point to a point θ to a point in question is θ' equal to the likelihood ratio $\frac{P(X|\theta)P(\theta)}{P(X|\theta')P(\theta')}$

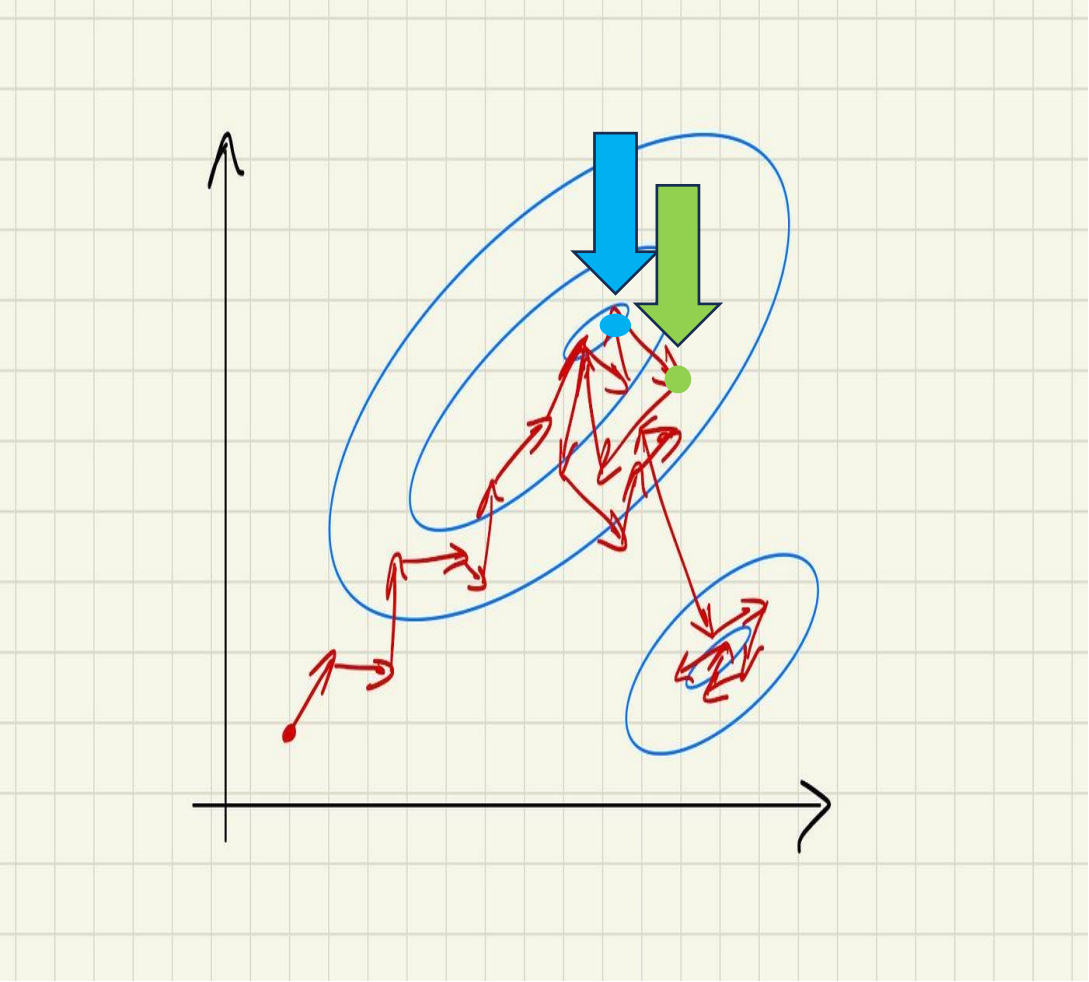
- Since the probability of being at any given point is proportional to the joint distribution, this algorithm samples from the posterior

MCMC in a Nutshell: $P(\theta|X) \propto P(X|\theta)P(\theta)$



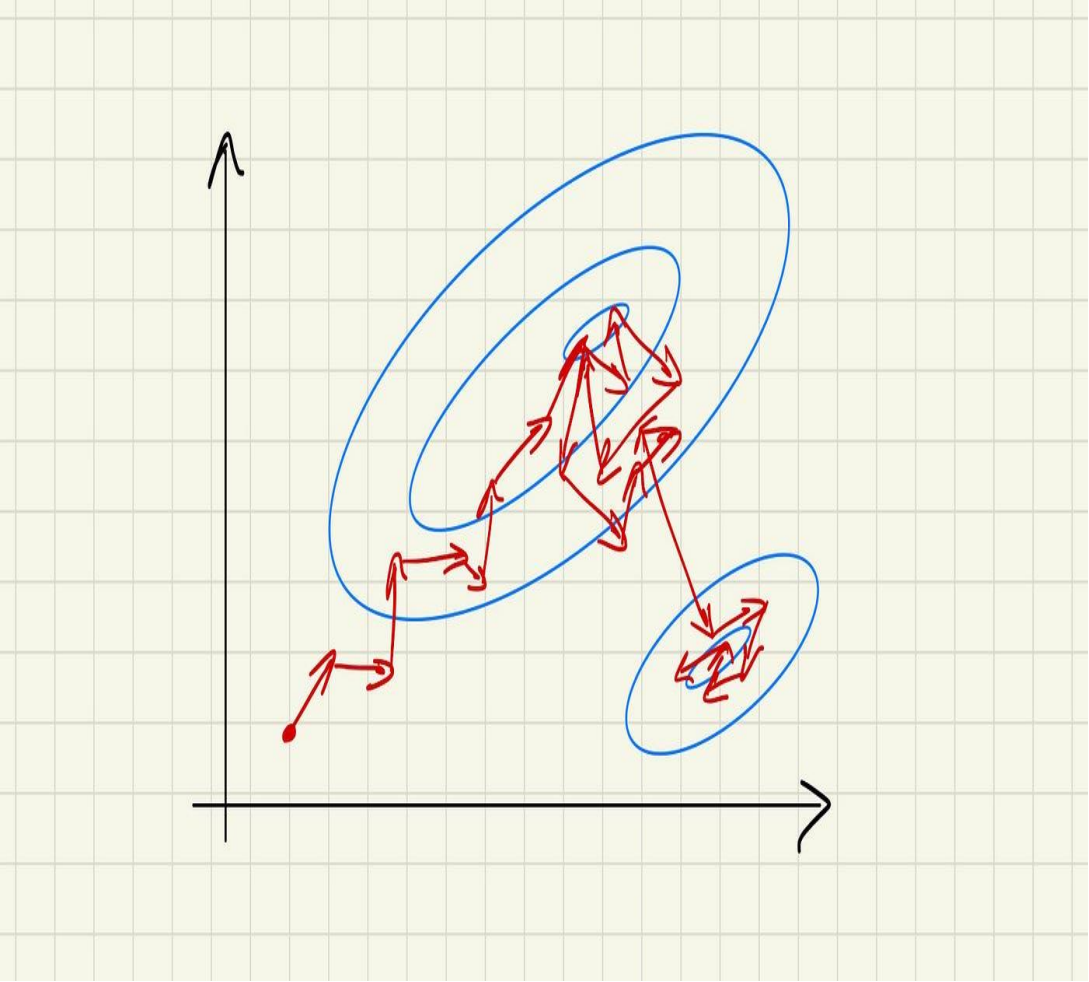
- You are at the blue point
- You want to move to the green point
- The blue point has a unnormalized joint probability, $P(X|\theta)P(\theta)$, of 1.5
- The green point has a unnormalized joint probability, $P(X|\theta')P(\theta')$, of 1.0

$$P(\theta|X) \propto P(X|\theta)P(\theta)$$



- You will move to the green point with a probability equal to the likelihood ratio: 1/1.5 = 2/3
  - The other 1/3 you stay put and visit your current point again
- In this way, you will visit each point in proportion to their unnormalized joint probability, $P(X|\theta)P(\theta)$, sampling from the posterior, $P(\theta|X)$, without having to calculate $P(X)$

Problems with MCMC



- Difficulty dealing with multimodal and unidentified distributions
- Over short time intervals samples exhibit serial correlation
- cf Bayesian Estimation of DSGE Models (Herbst and Schorfheide)
- SNPE can handle both of these

An Explanation of SNPE

SNPE Toy Model: $X = f(\theta, \epsilon)$

f could be as complex as a HA model or as simple as $X = \theta + \epsilon$

- X^* = data, X= model simulations, $p(\theta)$ = prior over θ, $\epsilon \sim N(\bar{\mathbf{0}}, \bar{\mathbf{1}})$

- Objective is to approximate posterior: $p(\theta | X = X^*)$

- **Step 1:** Simulate $\{X_i, \theta_i\}_{i=1}^{I}$ from joint distribution $\mathrm{p}(\theta, X)$

$$P(\theta|X) = \frac{P(X|\theta)P(\theta)}{P(X)}$$

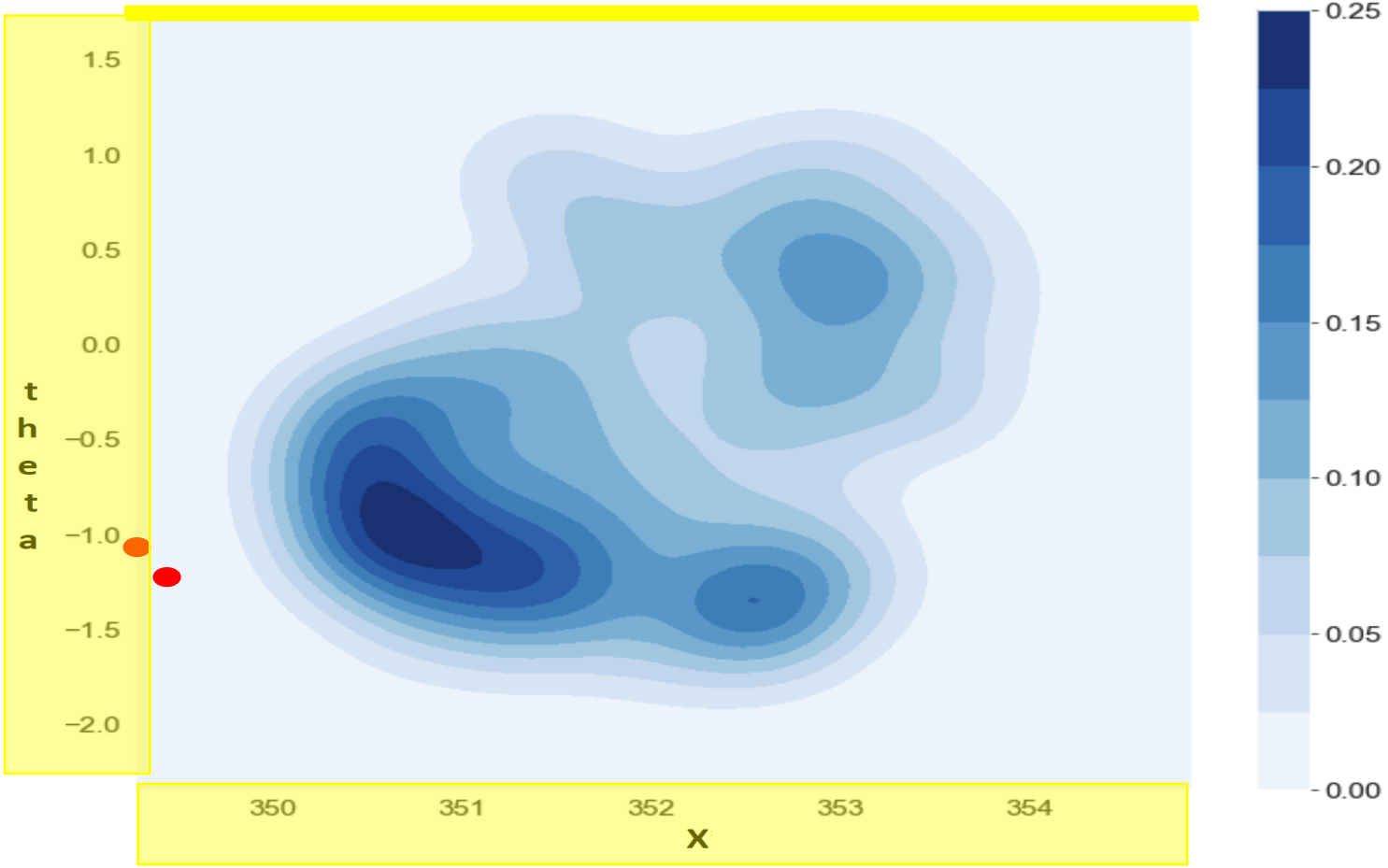
Simulate diagram

1. Sample θ_i from $p(\theta)$
2. Sample X_i from simulated model conditional on θ_i, $X_i = f(\theta_i, \epsilon)$

After this step, no need for the model

SNVI SNPE

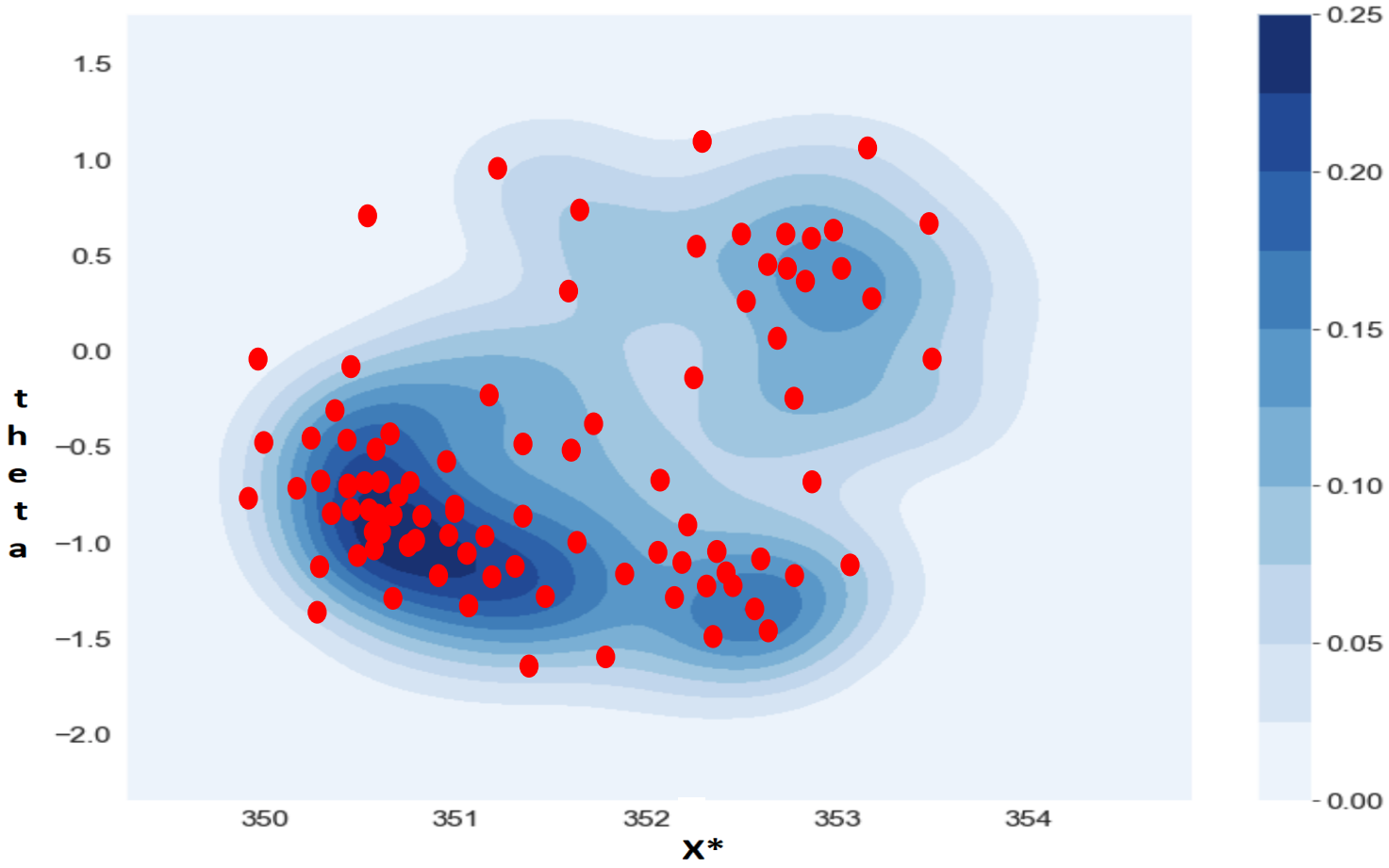
Step 1: Sample $\{X_i, \theta_i\}_{i=1}^{I} \sim p(\theta, X)$ via Monte Carlo Simulation



Back

Samples from the joint distribution :

$$\theta, X \sim P(\theta, X) = P(X|\theta)P(\theta)$$



Back

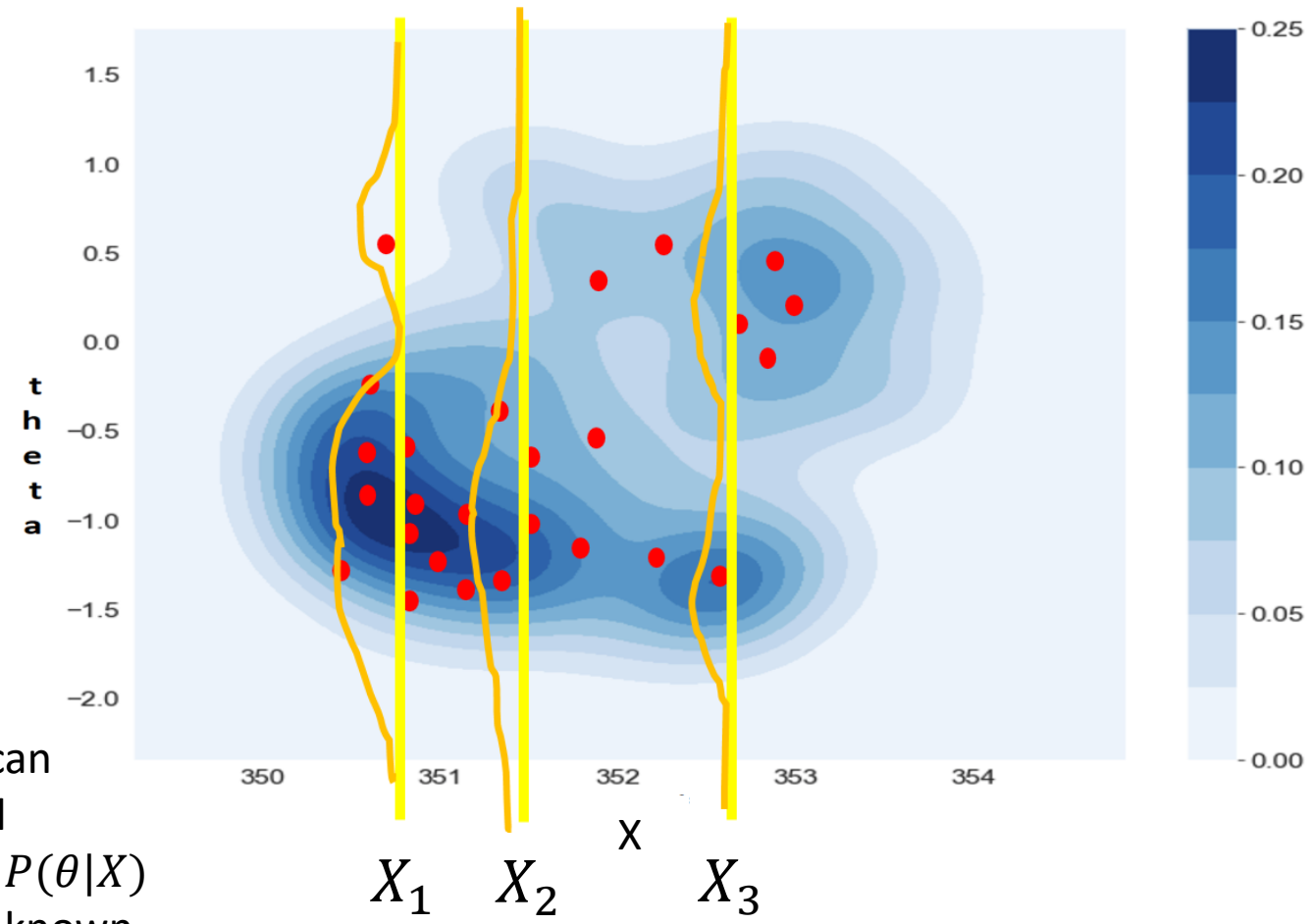
SNPE Toy Model (Step 2):

$$X = f(\theta, \epsilon)$$

- Using the joint samples, $\{X_i, \theta_i\}_{i=1}^{I}$, from step 1…
- **Step 2:** Estimate conditional density $p(\theta|X)$ on joint samples $\{X_i, \theta_i\}_{i=1}^{I}$!!!

Estimate diagram

Estimating the Conditional Density: $P(\theta|X)$



- From samples θ, X, we can estimate the conditional probability distribution: $P(\theta|X)$
  - Contour plot is not known to the econometrician

SNPE Toy Model (Step 3)

$$X = f(\theta, \epsilon)$$

- Using the conditional density estimator, $p(\theta|X)$, trained in step 2…

- **Step 3:** Condition X on real data, X^*, → posterior: $p(\theta|X = X^*)$

Condition diagram

Digging Deeper into Step 2: Conditional Density Estimation

SNPE Toy Model Redux: $\mathrm{X} = f(\theta, \epsilon)$

- **Step 1:** Simulate $\{X_i, \theta_i\}_{i=1}^{I}$ from joint distribution $\mathrm{p}(\theta, X)$

  Simulate diagram

- **Step 2:** Estimate conditional density $p(\theta|X)$ on joint samples $\{X_i, \theta_i\}_{i=1}^{I}$

  Estimate diagram

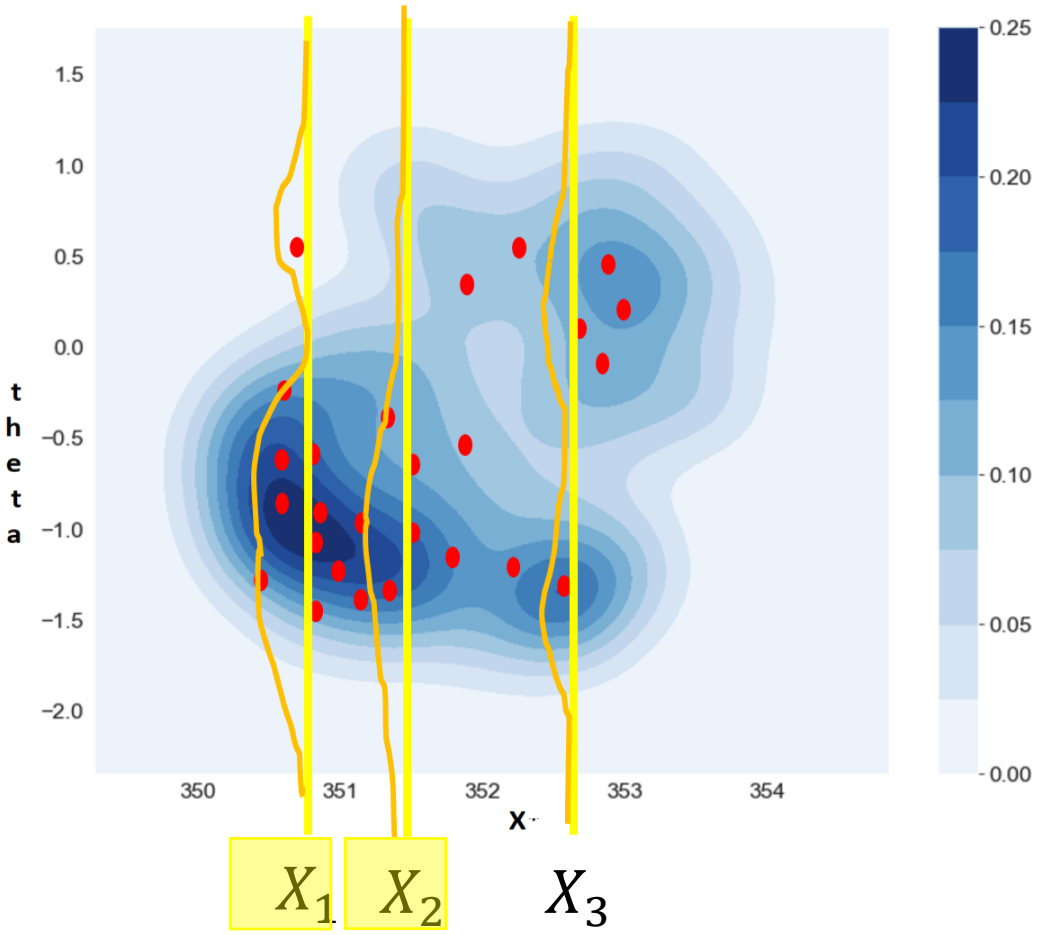
  - Other approaches (K & S + KMP) do steps 1 and 3 well, my method improves upon step 2

- **Step 3:** Condition X on the real data, X^*, forming the posterior: $p(\theta|X = X^*)$

  Condition diagram

Step 2: Estimate the conditional density $p(\theta|X)$

- High level: One way to estimate the conditional density is to build a parametric model that can evaluate p$(\theta|X)$ from joint samples and then maximize the conditional likelihood of p by updating its parameters
  - I will do this with a machine learning model: normalizing flows



Back

Unconditional Density Estimation with Normalizing Flows First

Some Syntax

- A normalizing flow is a chain of transformations: $\boldsymbol{Z_K} = f_{K-1} \circ f_{K-2} \circ \cdots f_0(\boldsymbol{Z_0})$, that take a take realizations from a known distribution, for example: $\boldsymbol{Z_0} \sim N(\overline{\boldsymbol{0}}, \overline{\boldsymbol{1}})$, and map it to realizations of the distribution, $\boldsymbol{Z_K}$, you want to estimate
  - $\boldsymbol{Z_k}$ (lower case k) is the J dimensional vector input to the kth J-to-J vector valued function f_k
- Since $\boldsymbol{Z_k}$ is a vector, superscripts, j, indicate the jth element of the vector:

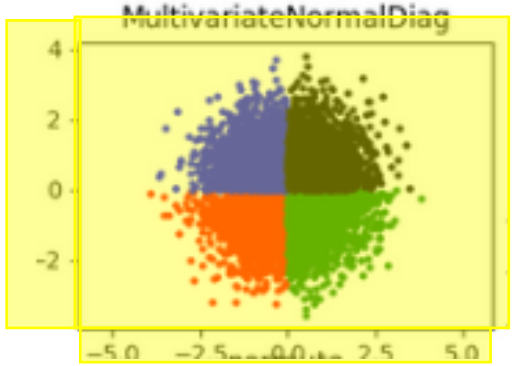
$$\boldsymbol{Z_k^j} \text{ is the jth element of the kth } \boldsymbol{Z} \text{ vector}$$

- For now, take $\boldsymbol{Z_k}$'s and f_k's as unrelated to θ and X
  - Later on, I will show how flows can perform conditional estimation of $\mathrm{P}(\theta|\mathrm{X})$

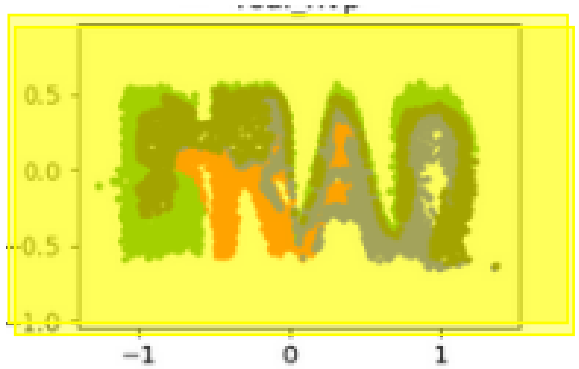
Normalizing Flow Main Takeaway

- At it’s core a normalizing code learns a mapping from realizations of a distribution that is known, to realizations of a distribution you want to estimate
- The mapping is created by a compositions of invertible functions
- Using the change of variables function, you can evaluate the likelihood of samples from the distribution you want to estimate
- Then you can modify the parameters of the normalizing flow to maximize the likelihood.

A Composition of Functions in a Flow allows it to become a Flexible Density Estimator



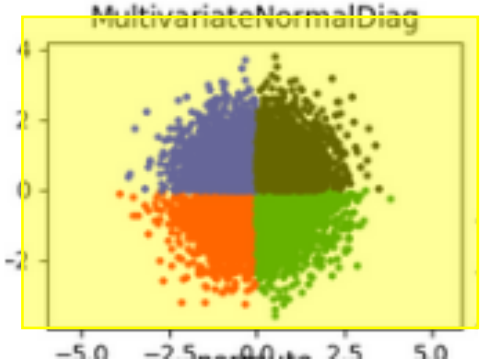
$$f_0(Z_0) \circ f_1 \circ f_2 \circ f_3 = Z_{K=4}$$



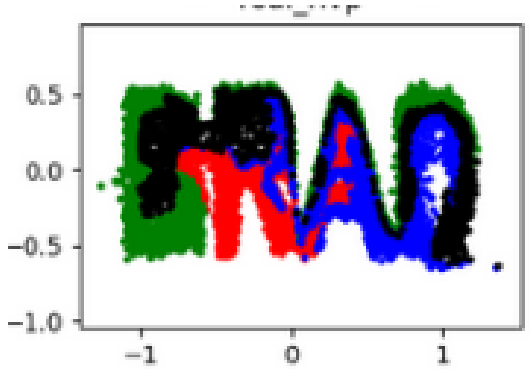
$\boldsymbol{Z_0} \sim N(\bar{0}, \bar{1})$

$\boldsymbol{Z_4}$

A Composition of Functions in a Flow allows it to become a Flexible Density Estimator



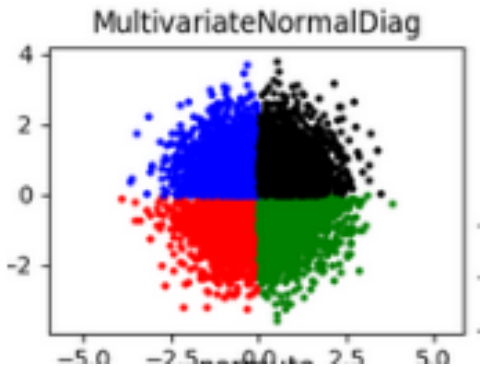
$$f_0(Z_0) \circ f_1 \circ f_2 \circ f_3 = Z_{K=4}$$

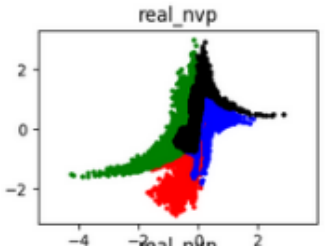


$\boldsymbol{Z_0} \sim N(\bar{0}, \bar{1})$

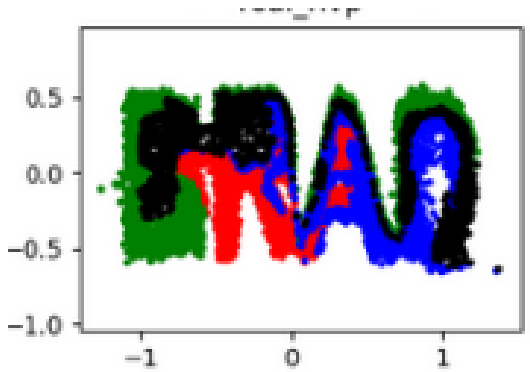
$\boldsymbol{Z_4}$

A Composition of Functions in a Flow allows it to become a Flexible Density Estimator





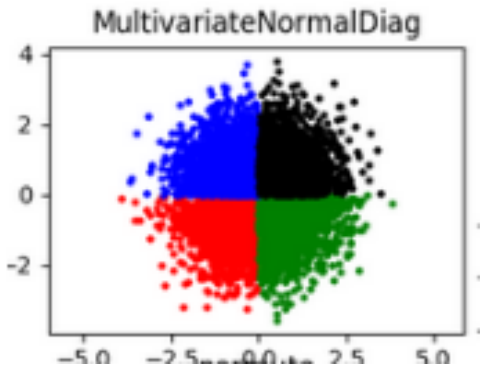
$$\boldsymbol{f_0}(\boldsymbol{Z_0}) \circ f_1 \circ f_2 \circ f_3 = Z_{K=4}$$

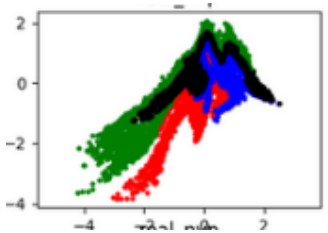


$\boldsymbol{Z_0} \sim N(\bar{0}, \bar{1})$

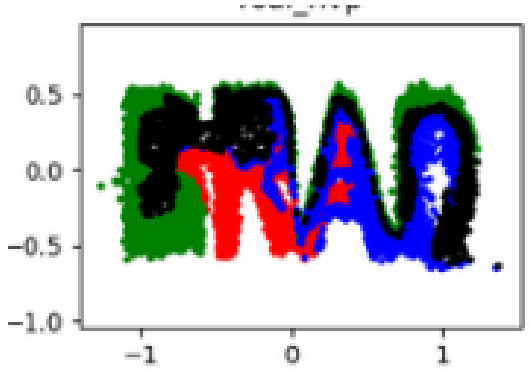
$\boldsymbol{Z_4}$

A Composition of Functions in a Flow allows it to become a Flexible Density Estimator





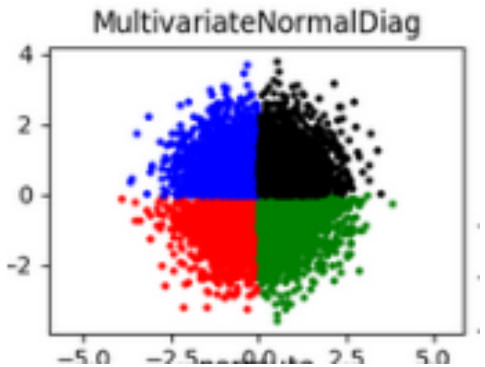
$$f_0(Z_0) \circ \boldsymbol{f_1} \circ f_2 \circ f_3 = Z_{K=4}$$

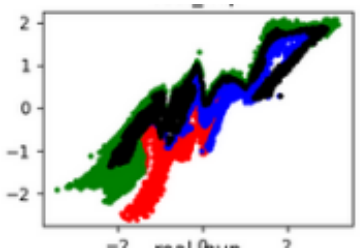


$\boldsymbol{Z_0} \sim N(\bar{0}, \bar{1})$

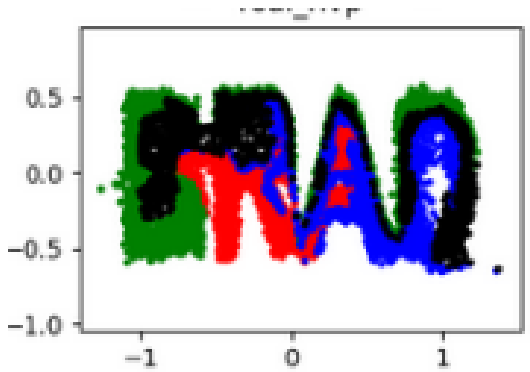
$\boldsymbol{Z_4}$

A Composition of Functions in a Flow allows it to become a Flexible Density Estimator





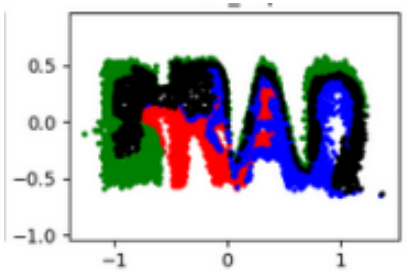
$$f_0(Z_0) \circ f_1 \circ \boldsymbol{f_2} \circ f_3 = Z_{K=4}$$

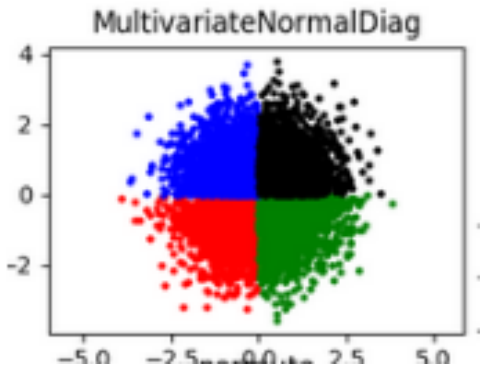


$\boldsymbol{Z_0} \sim N(\bar{0}, \bar{1})$

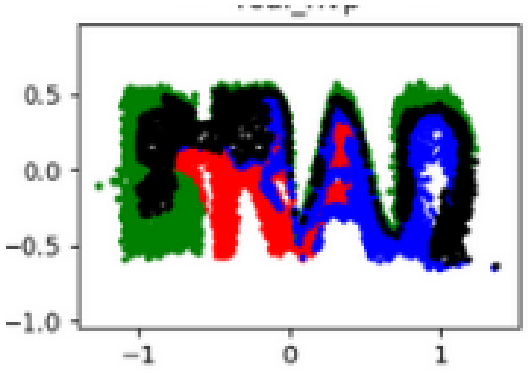
$\boldsymbol{Z_4}$

A Composition of Functions in a Flow allows it to become a Flexible Density Estimator





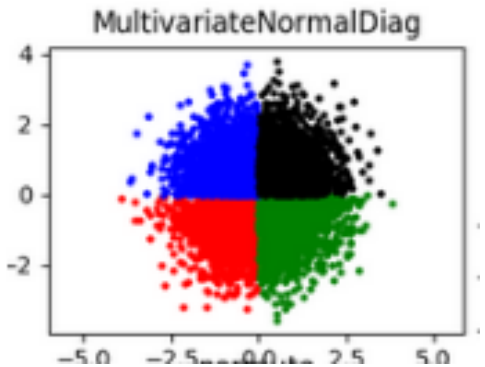
$$f_0(Z_0) \circ f_1 \circ f_2 \circ \boldsymbol{f_3} = \boldsymbol{Z_{K=4}}$$



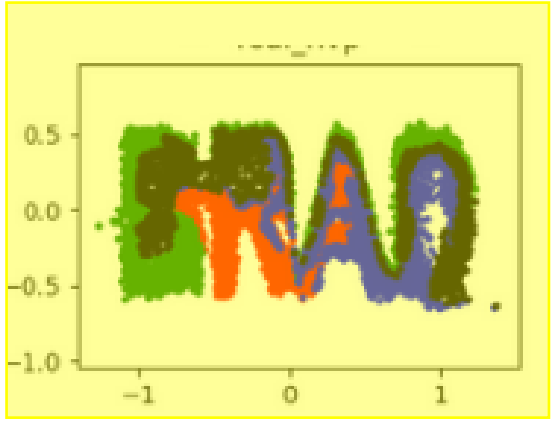
$\boldsymbol{Z_0} \sim N(\bar{0}, \bar{1})$

$\boldsymbol{Z_4}$

A Composition of Functions in a Flow allows it to become a Flexible Density Estimator



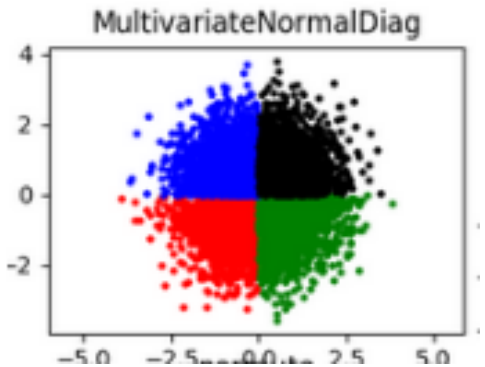
$$f_0(Z_0) \circ f_1 \circ f_2 \circ f_3 = Z_{K=4}$$



$\boldsymbol{Z_0} \sim N(\bar{0}, \bar{1})$

$\boldsymbol{Z_4}$

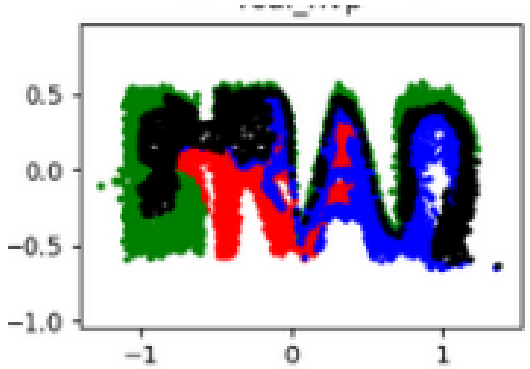
A Composition of Functions in a Flow allows it to become a Flexible Density Estimator



$$\boldsymbol{Z_0} \sim N(\bar{0}, \bar{1})$$

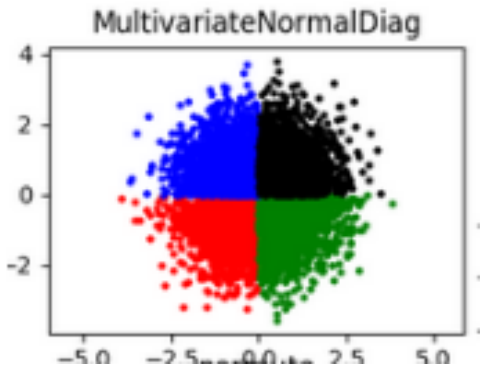
$$f_0(Z_0) \circ f_1 \circ f_2 \circ f_3 = Z_{K=4}$$

- Each f_k is a J by J invertible function
  - In this case J = 2

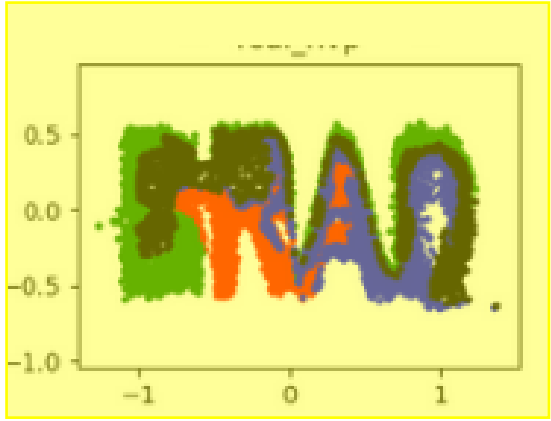


$$\boldsymbol{Z_4}$$

A Composition of Functions in a Flow allows it to become a Flexible Density Estimator



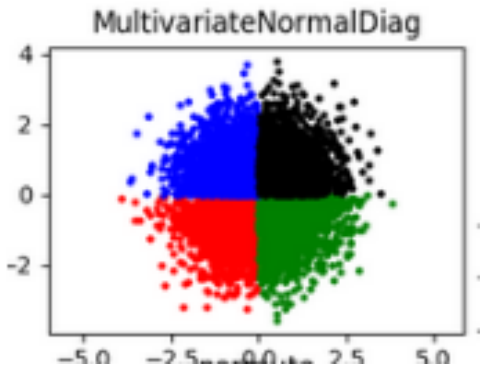
$$f_0(Z_0) \circ f_1 \circ f_2 \circ f_3 = Z_{K=4}$$



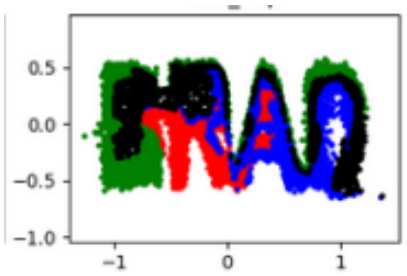
$\boldsymbol{Z_0} \sim N(\bar{0}, \bar{1})$

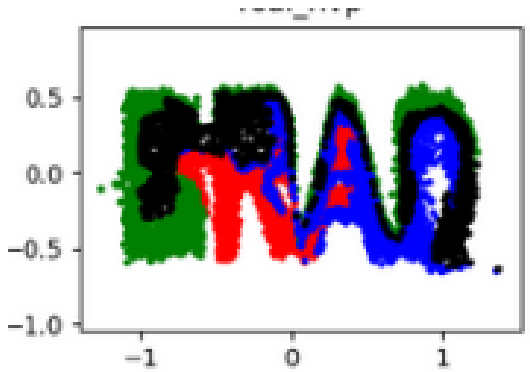
$\boldsymbol{Z_4}$

A Composition of Functions in a Flow allows it to become a Flexible Density Estimator



$$f_0(Z_0) \circ f_1 \circ f_2 \circ \boldsymbol{f_3^{-1}}(\boldsymbol{Z_{K=4}})$$

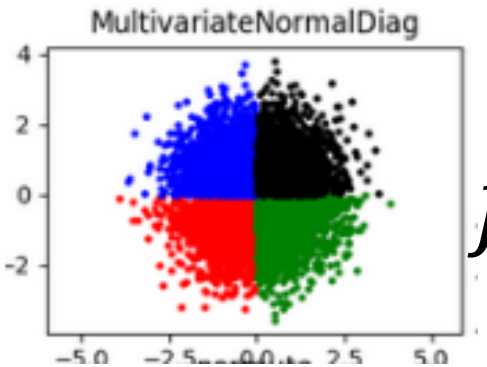


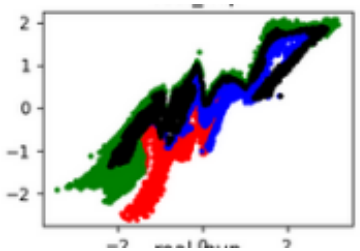


$\boldsymbol{Z_0} \sim N(\bar{0}, \bar{1})$

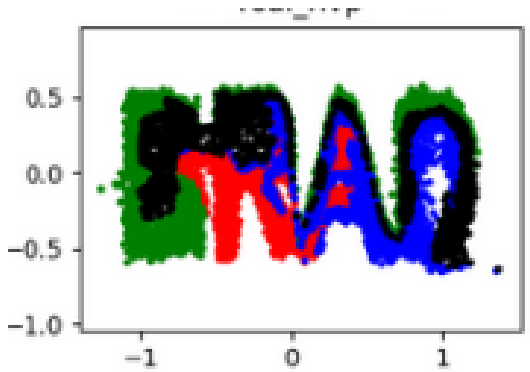
$\boldsymbol{Z_4}$

A Composition of Functions in a Flow allows it to become a Flexible Density Estimator





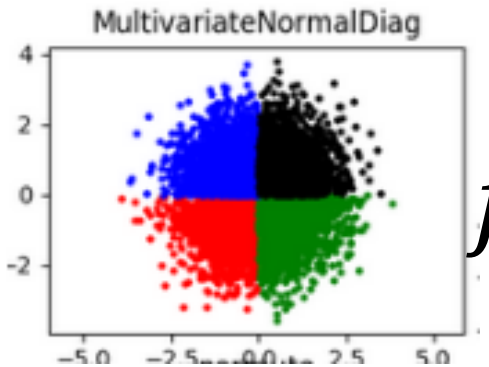
$$f_0(Z_0) \circ f_1 \circ \boldsymbol{f_2^{-1}} \circ f_3 = Z_{K=4}$$

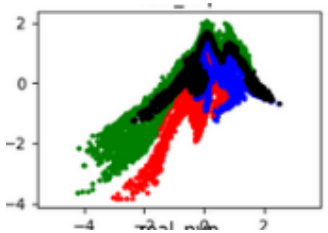


$\boldsymbol{Z_0} \sim N(\bar{0}, \bar{1})$

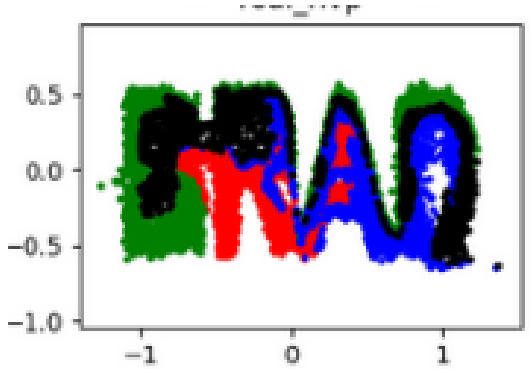
$\boldsymbol{Z_4}$

A Composition of Functions in a Flow allows it to become a Flexible Density Estimator





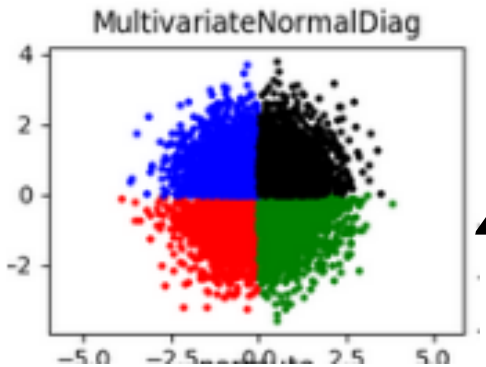
$$f_0(Z_0) \circ \boldsymbol{f_1^{-1}} \circ f_2 \circ f_3 = Z_{K=4}$$

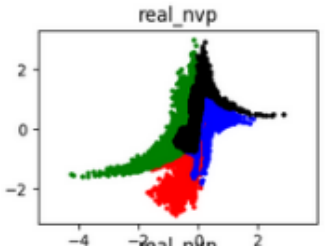


$\boldsymbol{Z_0} \sim N(\bar{0}, \bar{1})$

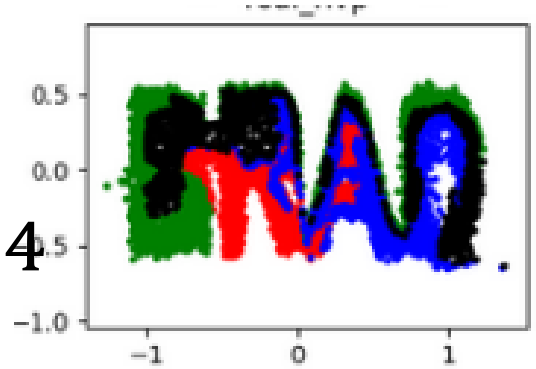
$\boldsymbol{Z_4}$

A Composition of Functions in a Flow allows it to become a Flexible Density Estimator





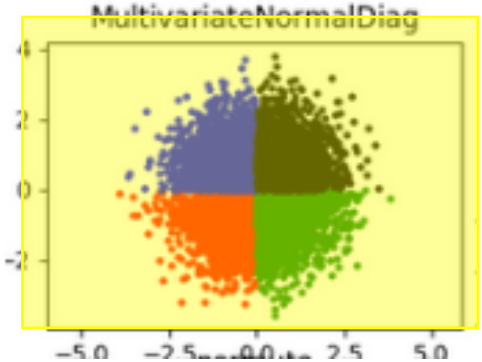
$$\boldsymbol{Z_0} = \boldsymbol{f_0^{-1}} \circ f_1 \circ f_2 \circ f_3 = Z_{K=4}$$



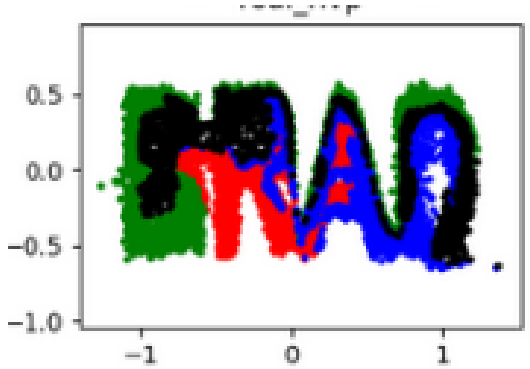
$\boldsymbol{Z_0} \sim N(\bar{0}, \bar{1})$

$\boldsymbol{Z_4}$

A Composition of Functions in a Flow allows it to become a Flexible Density Estimator



$$f_0(Z_0) \circ f_1 \circ f_2 \circ f_3 = Z_{K=4}$$



$\boldsymbol{Z_0} \sim N(\bar{0}, \bar{1})$

$\boldsymbol{Z_4}$

What is the Point of a Flow?

- Given points sampled from some unknown distribution, for example: $\boldsymbol{Z_K} \sim \mathrm{p}(\boldsymbol{Z_K})$, the normalizing flow will learn a mapping from $\boldsymbol{Z_0}$, the realizations of a known distribution, to $\boldsymbol{Z_K}$

- The flow will do this by optimizing the likelihood with respect to the yet to be discussed parameters of f_k's

- Thus, we need to figure out 2 things:

  - How to calculate the likelihood

  - What are the parameters of the f_k's

Fundamental Insight Regarding the Likelihood: A Change of Variables

- If $\boldsymbol{Z_0} \sim q(\boldsymbol{Z_0})$ is a known J-dimensional density, say $N(\overline{\mathbf{0}}, \overline{\mathbf{1}}\ )$,
  - Let f be a J-to-J dimensional function, then $f(\boldsymbol{Z_0})$ has the density:

$$\mathrm{p}\big(f(\boldsymbol{Z_0})\big) = \mathrm{q}(\boldsymbol{Z_0}) \left|\det \frac{\delta f}{\delta \boldsymbol{Z_0}}\right|^{-1}$$

- For a set of J-to-J dimensional f's: $\boldsymbol{Z_K} = f_{K-1} \circ f_{K-2} \circ \cdots f_0(\boldsymbol{Z_0})$:
  - Even irregular distribution samples, $\boldsymbol{Z_K}$, can be matched and have a log likelihood of:

$$\ln\big(\mathrm{p}(\boldsymbol{Z_K})\big) = \ln\big(\mathrm{q}(\boldsymbol{Z_0})\big) - \sum_{k=0}^{k=K-1} \ln(|\det \frac{\delta \boldsymbol{f_k}}{\delta \boldsymbol{Z_k}}|)$$

- Now we know how to evaluate a likelihood for a normalizing flow

Constructing a Good $\mathrm{f}_{\boldsymbol{k}}$ layer

- Now to discuss the parameters of the f_k's

- For example, I will talk about a specific f_k

  - f_k takes in a J dimensional vector $\boldsymbol{Z_k}$ (lower case) and outputs a J dimensional vector $\boldsymbol{Z_{k+1}}$

Transform Structure

- Take J = 3, here is one parametrization of f_k that is invertible and has conditional dependence across Z^1, Z^2, and Z^3 for all k:

$$Z_{k+1}^1 = a_k^1 * Z_k^1 + b_k^1$$
$$Z_{k+1}^2 = a_k^2(Z_k^1) * Z_k^2 + b_k^2(Z_k^1)$$
$$Z_{k+1}^3 = a_k^3(Z_k^1, Z_k^2) * Z_k^3 + b_k^3(Z_k^1, Z_k^2)$$

- Couple things to talk about:
  - $a_k^2(Z_k^1)$, $b_k^2(Z_k^1)$, $a_k^3(Z_k^1, Z_k^2)$, and $b_k^3(Z_k^1, Z_k^2)$ are all functions (as will a_k^1 and b_k^1 when performing conditional estimation) → how to structure them
  - f_k which is a function from $\boldsymbol{Z_k}$ to $\boldsymbol{Z_{k+1}}$ is invertible

Transform Structure

- Take j = 3, here is one parametrization of f_k that is invertible and has conditional dependence across Z^1, Z^2, and Z^3 for all k:

$$Z_{k+1}^1 = a_k^1 * Z_k^1 + b_k^1$$
$$Z_{k+1}^2 = a_k^2(Z_k^1) * Z_k^2 + b_k^2(Z_k^1)$$
$$Z_{k+1}^3 = a_k^3(Z_k^1, Z_k^2) * Z_k^3 + b_k^3(Z_k^1, Z_k^2)$$

- Couple things to talk about:
  - $a_k^2(Z_k^1)$, $b_k^2(Z_k^1)$, $a_k^3(Z_k^1, Z_k^2)$, and $b_k^3(Z_k^1, Z_k^2)$ are all functions (as will a_k^1 and b_k^1 when performing conditional estimation) → how to structure them
  - f_k which is a function from $\boldsymbol{Z_k}$ to $\boldsymbol{Z_{k+1}}$ is invertible

a_k and b_k Functions

- In theory, a_k and b_k can be any function with optimizable parameters…
- …however, in practice, a_k and b_k are outputs of a feed-forward neural network
- If necessary, for intuition, replace feed-forward neural network with a high dimensional polynomial regression
- Thus, the likelihood can be maximized by updating the parameters of the a_k and b_k functions in each transform f_k
  - This will learn the correct mapping between the realizations of a known $\boldsymbol{Z_0}$ distribution like samples from $N(\mathbf{0}, \mathbf{1})$ and the target distribution $\boldsymbol{Z_K}$

Transform Structure

- Take j = 3, here is one parametrization of f_k that is invertible and has conditional dependence across Z^1, Z^2, and Z^3 for all k:

$$Z_{k+1}^1 = a_k^1 * Z_k^1 + b_k^1$$
$$Z_{k+1}^2 = a_k^2(Z_k^1) * Z_k^2 + b_k^2(Z_k^1)$$
$$Z_{k+1}^3 = a_k^3(Z_k^1, Z_k^2) * Z_k^3 + b_k^3(Z_k^1, Z_k^2)$$

- Couple things to talk about:
  - $a_k^2(Z_k^1)$, $b_k^2(Z_k^1)$, $a_k^3(Z_k^1, Z_k^2)$, and $b_k^3(Z_k^1, Z_k^2)$ are all functions (as will a_k^1 and b_k^1 when performing conditional estimation) → how to structure them
  - f_k which is a function from $\boldsymbol{Z_k}$ to $\boldsymbol{Z_{k+1}}$ is invertible

Invertibility from Z_k to Z_{k+1}

- Take j = 3, here is one parametrization of f_k that is invertible and has conditional dependence across Z^1, Z^2, and Z^3 for all k:

$$Z_{k+1}^1 = a_k^1 * Z_k^1 + b_k^1$$
$$Z_{k+1}^2 = a_k^2(Z_k^1) * Z_k^2 + b_k^2(Z_k^1)$$
$$Z_{k+1}^3 = a_k^3(Z_k^1, Z_k^2) * Z_k^3 + b_k^3(Z_k^1, Z_k^2)$$

Invertibility from Z_{k+1} to Z_k

- Using the same f_k as earlier slides:

We know the elements of Z_{k+1}

$$Z_{k+1}^1 = a_k^1 * Z_k^1 + b_k^1$$
$$Z_{k+1}^2 = a_k^2(Z_k^1) * Z_k^2 + b_k^2(Z_k^1)$$
$$Z_{k+1}^3 = a_k^3(Z_k^1, Z_k^2) * Z_k^3 + b_k^3(Z_k^1, Z_k^2)$$

Attempting to solve for elements of Z_k

- Knowing Z_{k+1}^1, can invert the affine first equation to get Z_k^1:

$$Z_k^1 = \frac{Z_{k+1}^1 - b_k^1}{a_k^1}$$

- Then:

$$Z_k^2 = \frac{Z_{k+1}^2 - b_k^2(Z_k^1)}{a_k^2(Z_k^1)}$$

- Finally, $a_k^3(Z_k^1, Z_k^2)$ and $b_k^3(Z_k^1, Z_k^2)$ are functions of Z_k^1, Z_k^2, so Z_k^3 can be solved
  - Proof with J > 3 is via induction

Additional Details

- Z_k^1 is not conditioned on any variables and Z_k^3 is conditioned on all the variables
  - To maintain as much conditionality as possible, for all k, the order of the Z_k vectored should be randomly permuted in each layer
- We now have a flow $\boldsymbol{Z_K} = f_{K-1} \circ f_{K-2} \circ \cdots f_0(\boldsymbol{Z_0})$, that can be density estimated by modifying the parameter of each f_k to maximize the likelihood:

$$\ln\big(\mathrm{p}(\boldsymbol{Z_K})\big) = \ln\big(\mathrm{q}(\boldsymbol{Z_0})\big) - \sum_{k=0}^{k=K-1} \ln(|\det \frac{\delta \boldsymbol{f_k}}{\delta \boldsymbol{Z_k}}|)$$

Unconditional to Conditional Normalizing Flow

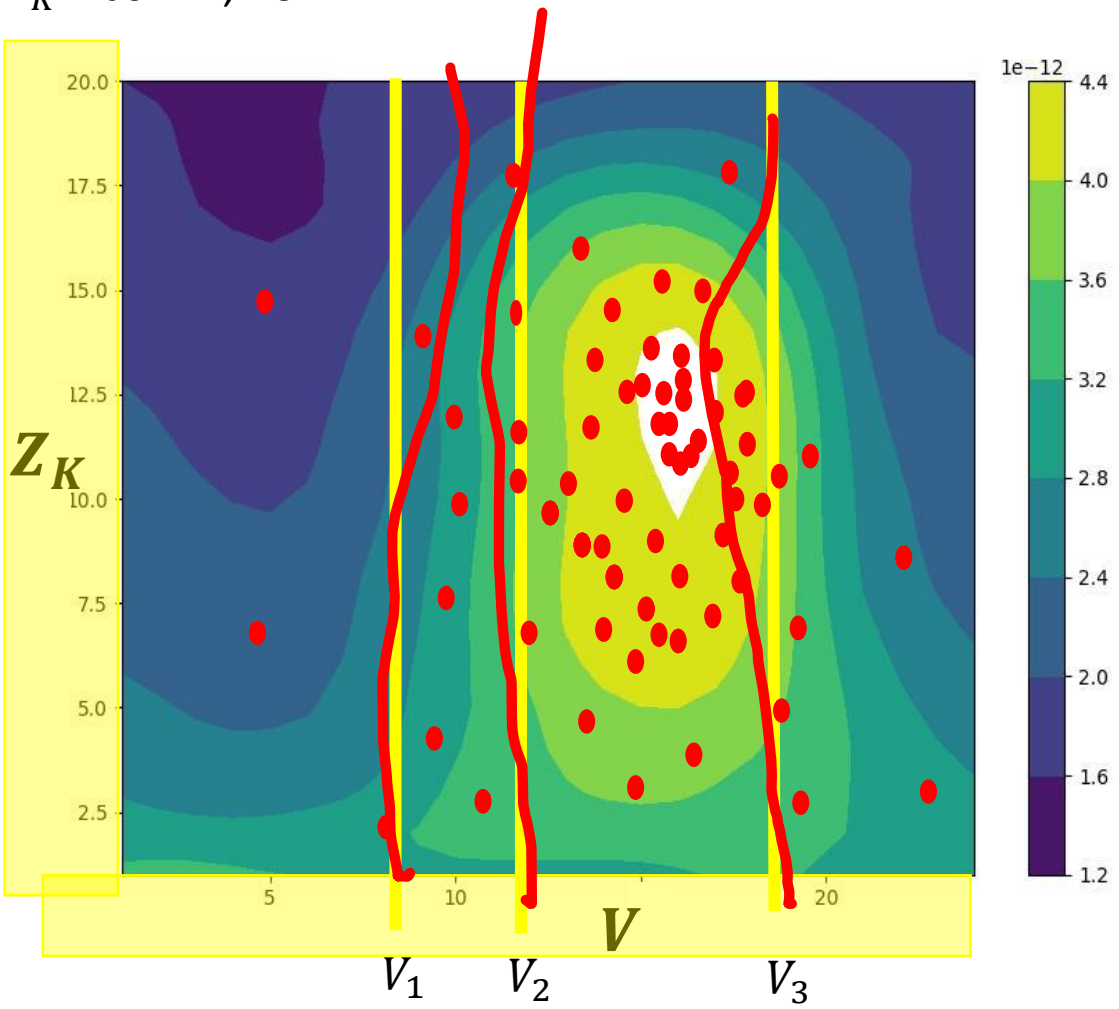
Previously Z_K was 2-D, now 1-D

- To condition the flow on Y take each:

$$Z_{k+1}^{j} = a_k^j\left(Z_k^1 \dots Z_k^{j-1}\right) * Z_k^j + b_k^j\,(Z_k^1 \dots Z_k^{j-1}) \dots$$

to

$$Z_{k+1}^{j} = a_k^j(Z_k^1 \dots Z_k^{j-1}, V_1) * Z_k^j + b_k^j\,(Z_k^1 \dots Z_k^{j-1}, V_1)$$



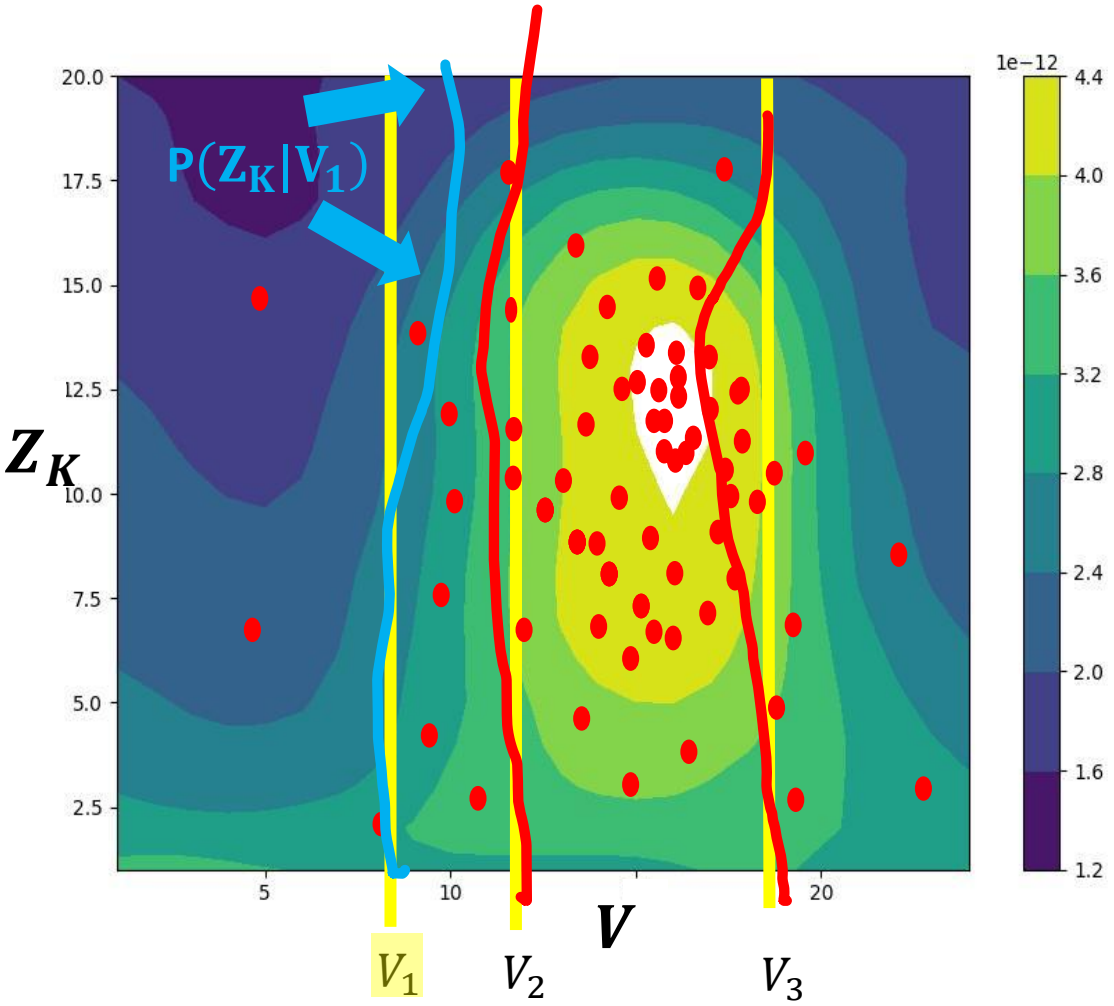
Unconditional to Conditional Normalizing Flow

- To condition the flow on Y take each:

$$Z_{k+1}^{j} = a_k^j(Z_k^1 \dots Z_k^{j-1}) * Z_k^j + b_k^j (Z_k^1 \dots Z_k^{j-1}) \dots$$

to

$$Z_{k+1}^{j} = a_k^j(Z_k^1 \dots Z_k^{j-1}, V_1) * Z_k^j + b_k^j (Z_k^1 \dots Z_k^{j-1}, V_1)$$



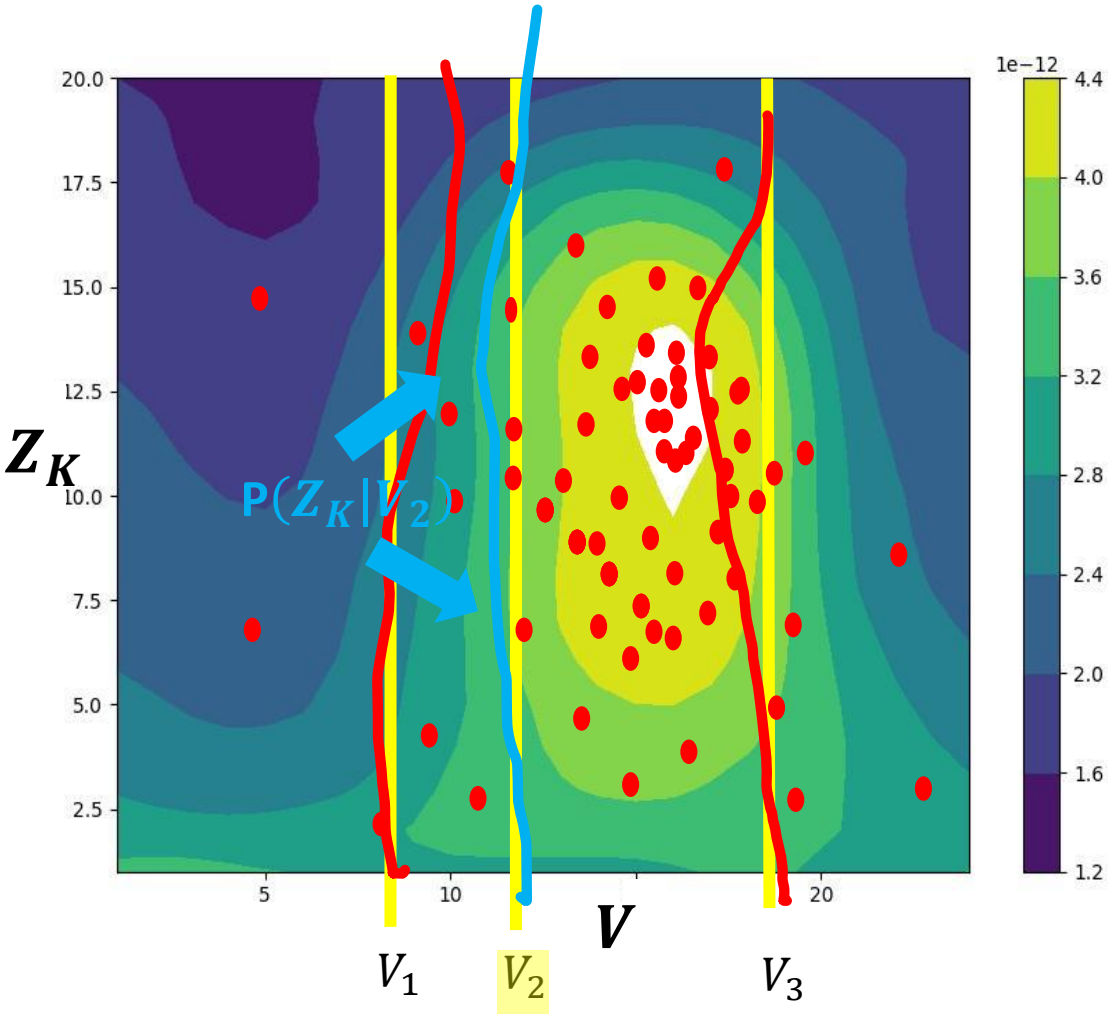
Unconditional to Conditional Normalizing Flow

- To condition the flow on Y take each:

$$Z_{k+1}^{j} = a_k^j(Z_k^1 \dots Z_k^{j-1}) * Z_k^j + b_k^j (Z_k^1 \dots Z_k^{j-1}) \dots$$

to

$$Z_{k+1}^{j} = a_k^j(Z_k^1 \dots Z_k^{j-1}, V_2) * Z_k^j + b_k^j (Z_k^1 \dots Z_k^{j-1}, V_2)$$



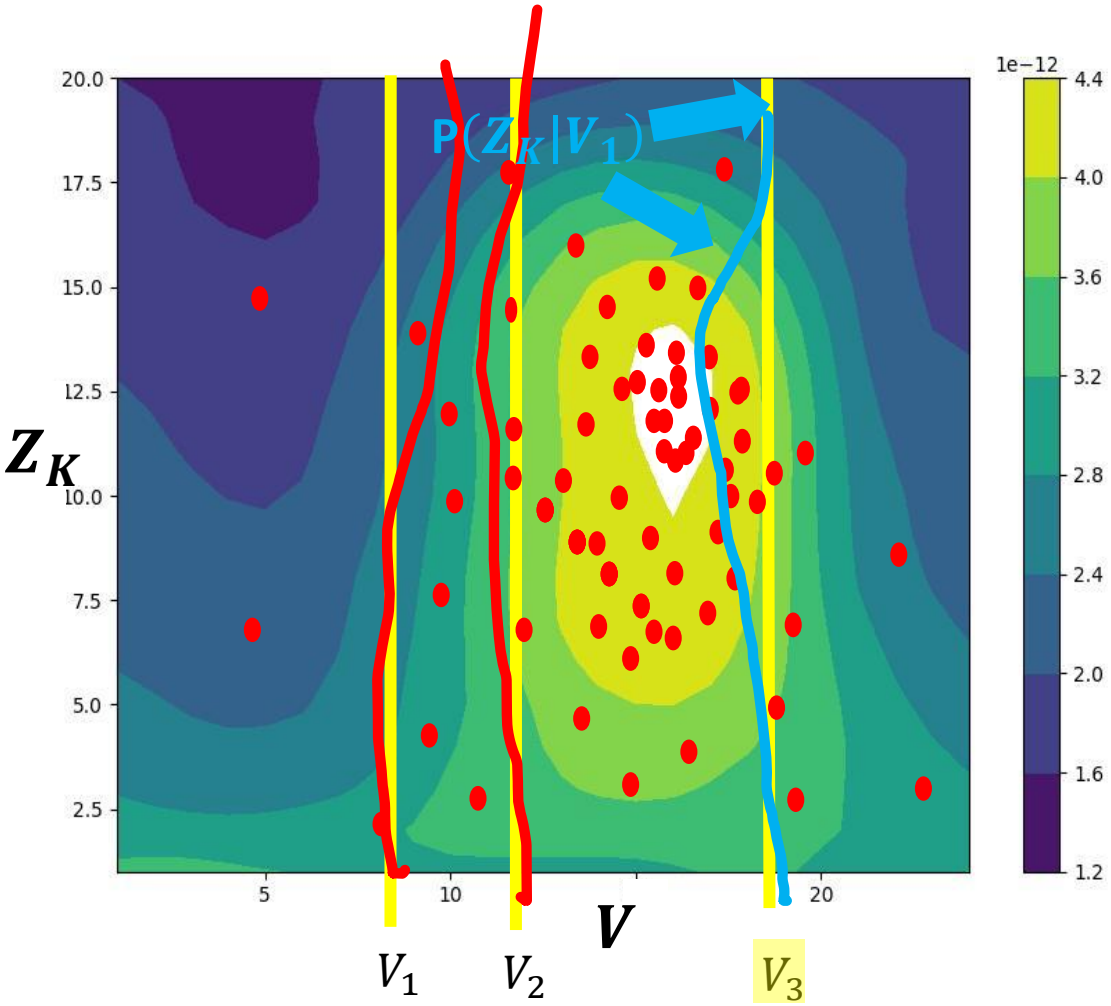
Unconditional to Conditional Normalizing Flow

- To condition the flow on Y take each:

$$Z_{k+1}^{j} = a_k^j(Z_k^0, Z_k^1 \dots Z_k^{j-1}) * Z_k^j + b_k^j\,(Z_k^0, Z_k^1 \dots Z_k^{j-1}) \dots$$

to

$$Z_{k+1}^{j} = a_k^j(Z_k^1 \dots Z_k^{j-1}, V_3) * Z_k^j + b_k^j\,(Z_k^1 \dots Z_k^{j-1}, V_3)$$



Maximum Likelihood Estimation

- When estimating the distribution of $\boldsymbol{\theta}$ from $\boldsymbol{X}$, set $\boldsymbol{Z_K} = \boldsymbol{\theta}$ and $\boldsymbol{Y} = \boldsymbol{X}$, and learn a mapping from $\boldsymbol{Z_0}$ to $\boldsymbol{Z_K} = \boldsymbol{\theta}$ conditional on $\boldsymbol{X}$, via $\{X_i, \theta_i\}_{i=1}^{I}$ samples:

$$\ln\big(\mathrm{p}(\boldsymbol{Z_K} = \boldsymbol{\theta}|\boldsymbol{V} = \boldsymbol{X})\big) = \ln\big(\mathrm{q}(\boldsymbol{Z_0})\big) - \sum_{k=0}^{K-1} \ln(|\det \frac{\delta \boldsymbol{f_k}|\boldsymbol{X}}{\delta \boldsymbol{Z_k}}|)$$

- Can perform maximum likelihood via:

$$\max_{\text{wrt params of } a_k^j,\, b_k^j} \sum_{i}^{i=I} \ln(p(\{\boldsymbol{\theta}|\boldsymbol{X}\}_i))$$

Step 1: Simulate $\{X_i, \theta_i\}_{i=1}^{I}$ from joint distribution $\mathrm{p}(\theta, X)$

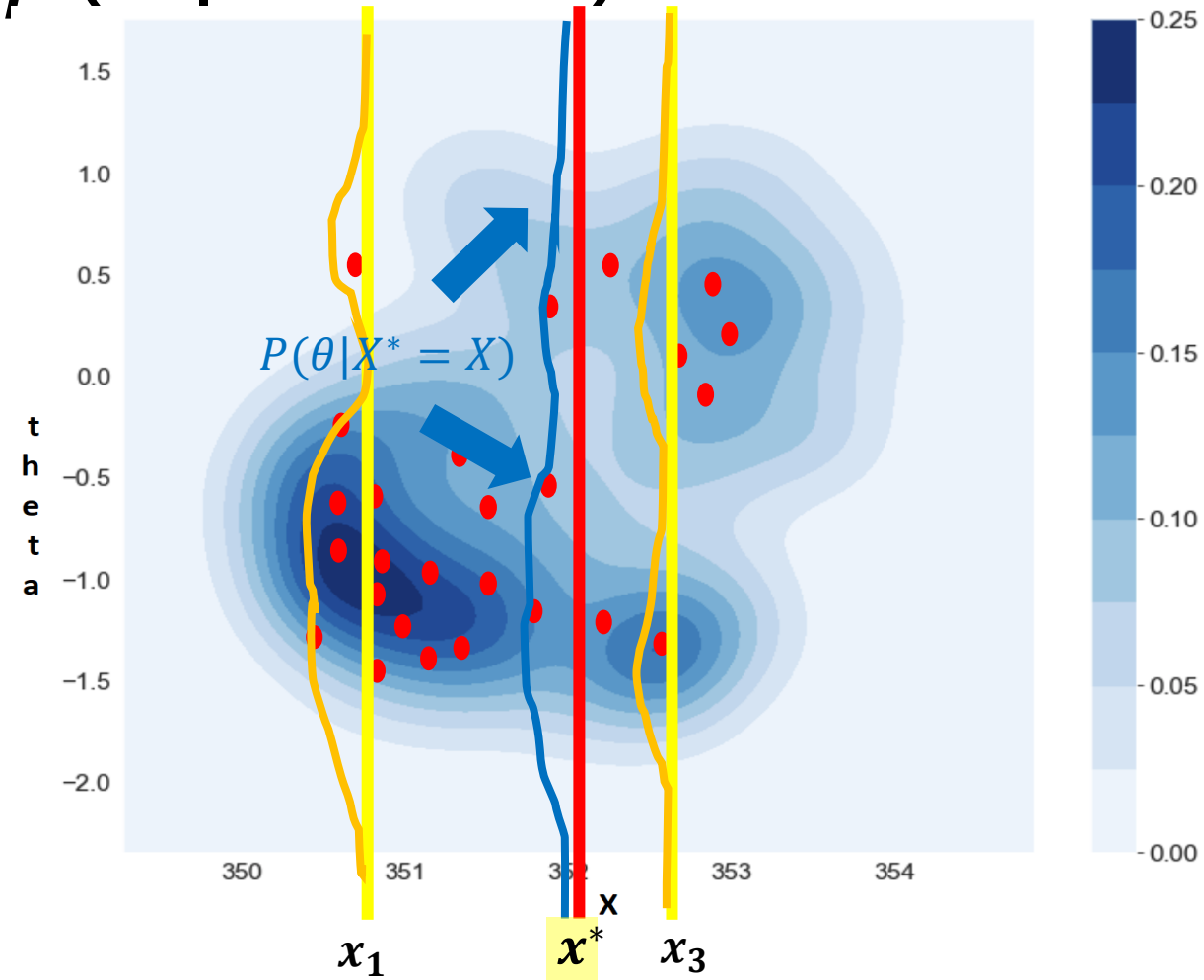
- a_k^j, b_k^j functions with parameters

Step 3: Condition X on the real data, X^*, Forming the Posterior: $p(\theta|X = X^*)$

- Now that you have $p(\theta|X)$
  - Random variable conditioned on generic X

- The posterior is:
  - $P(\theta|X = X^*)$

  - $p(\theta|X)$ is conditioned on the real-world particular data, X^*



Back Multi-Round Inference

Results

SNPE vs Bayesian Methods

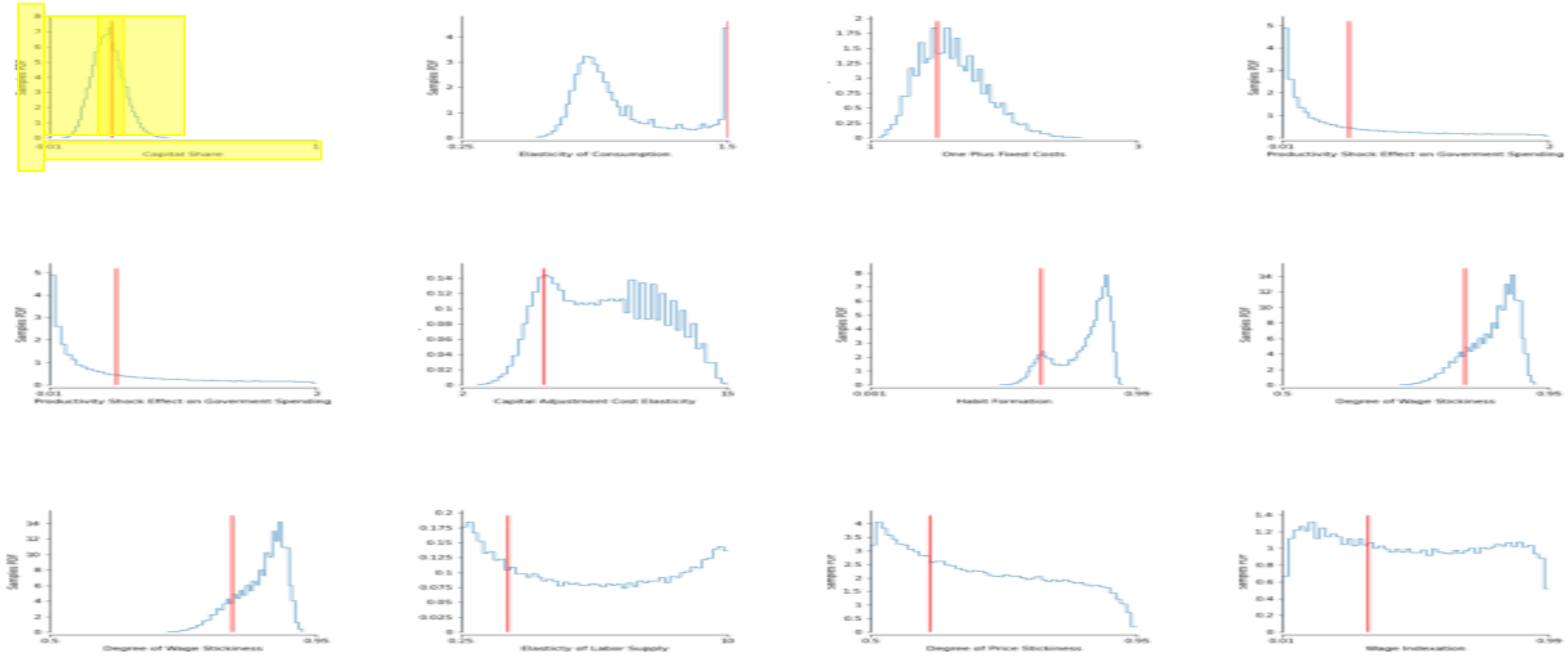
Symbol	Meaning
✓	Model can do this without difficulties
∼	Model comes close to or can with modifications
?	Couldn't find research
✗	Model cannot do this

Solution Method/Algorithm	SNPE	MCMC	SMC-est	ABC
Estimate small scale DSGE	✓	✓	✓	∼
Estimate medium/large DSGE	✓	✓	✓	∼
Estimate HANK - Reiter	✓	∼	∼	∼
Estimate HANK - Winberry	✓	✓	?	∼
Estimate without likelihood function	✓	∼	∼	✓
Quality posteriors even with large models	✓	✗	✓	∼

Perturbation Projection Value Function Iteration HANK Reiter HANK Winberry

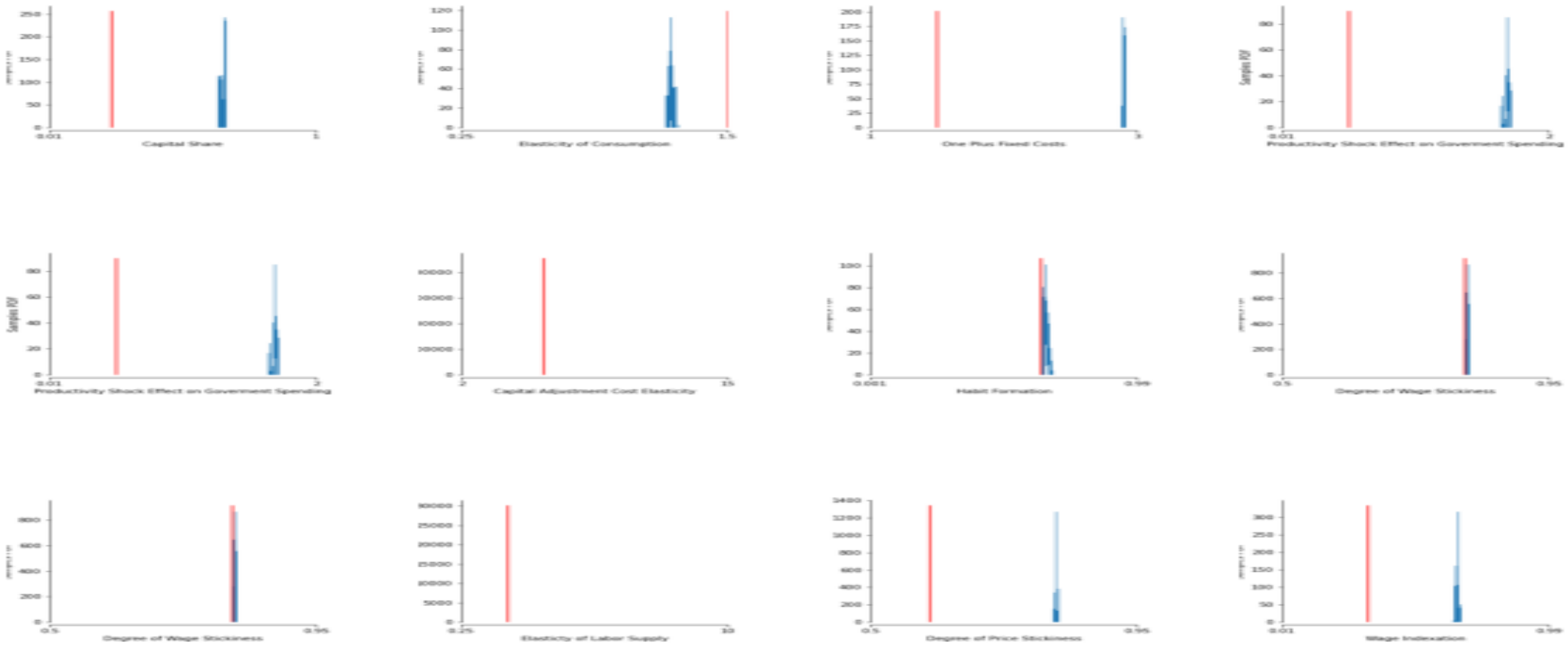
SNPE/MCMC Performance on Smets-Wouters

SNPE after Five Hundred Thousand Iterations (Uniform Prior)



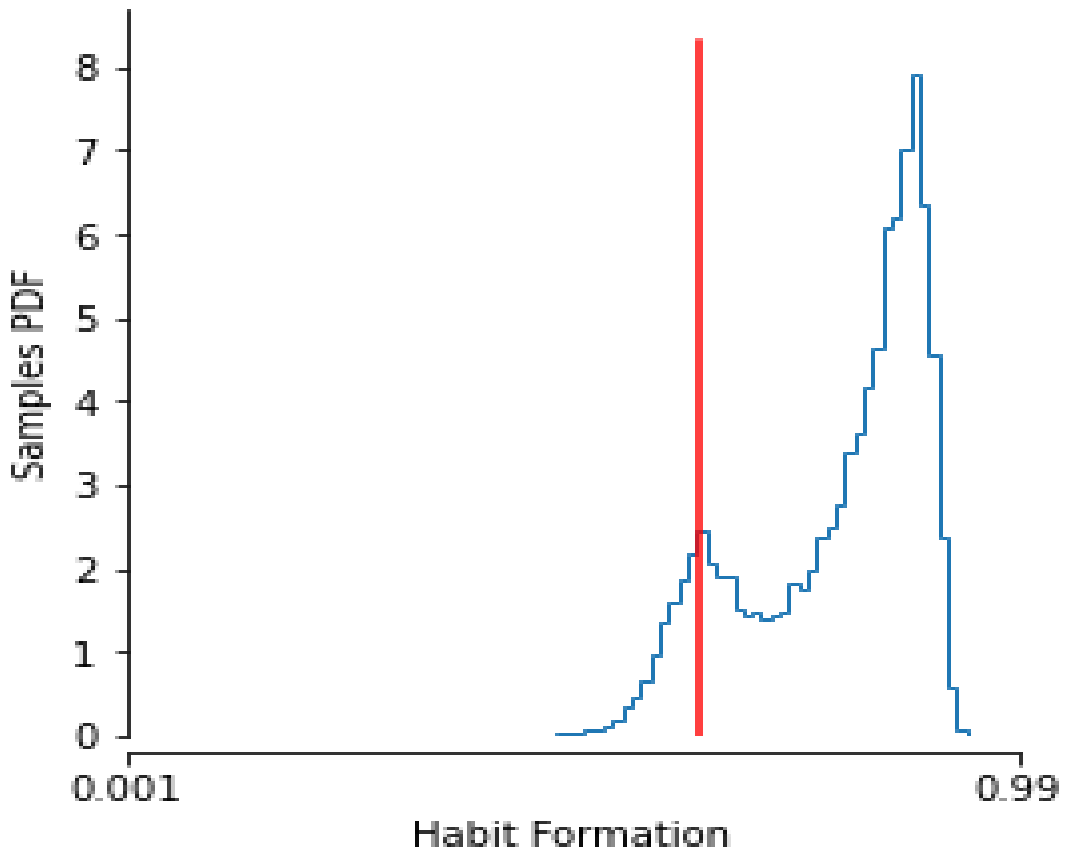
[Back]

Metropolis-Hastings MCMC after Ten Million Iterations (Uniform Prior)



[Back]

Theoretical Underpinning for SNPE Accuracy vs Monte Carlo



- Sampling from multimodal distributions with MCMC don't have polynomial computational guarantees (Chewi 2023)
- Using a parametric density estimator via maximum likelihood are roughly quadratic/polynomial in cost (Farrell 2021)

Back Chewi Intuition

SNPE Performance on Heterogenous Agent Models

Liu and Plagborg-Moller Estimation

- Following Liu and Plagborg-Moller (2023), likelihood estimate Krusell-Smith (KS) HA model with micro moments

- In this case, match output, consumption, investment and wage macro variables and match wealth as micro variables

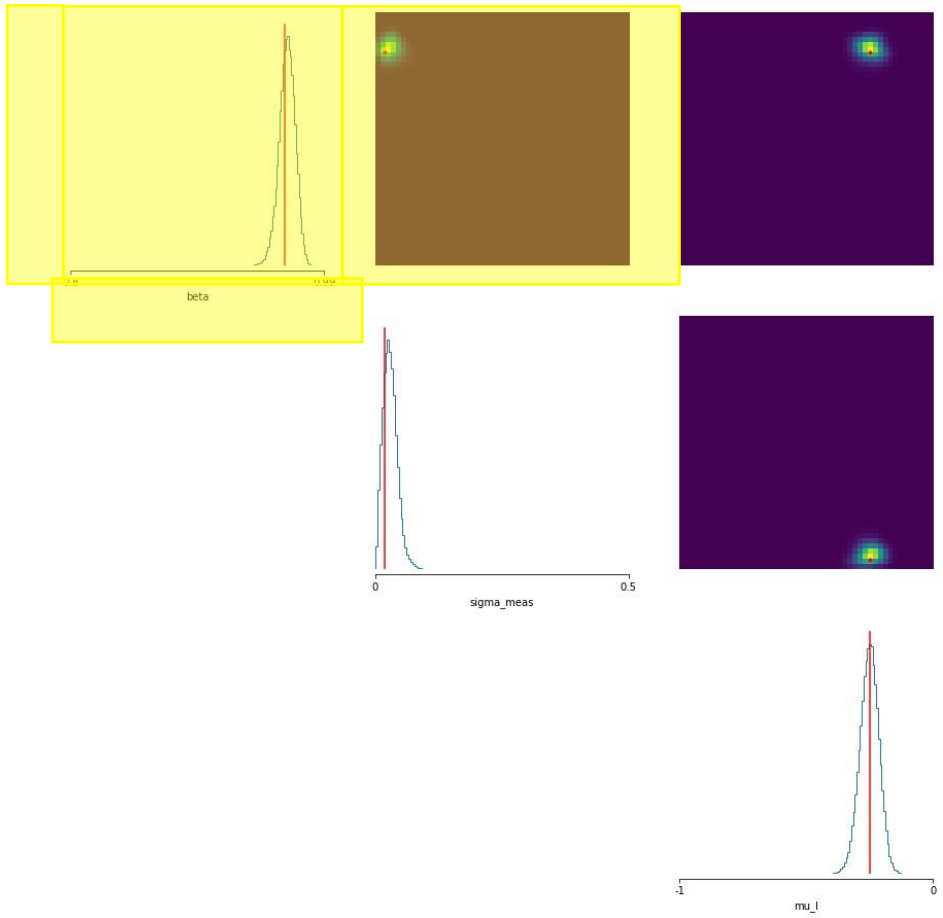
- Liu and Plagborg-Moller estimate by drawing from the state, macro variables conditional on the state, and micro variables conditional on state and macro and evaluating likelihood with measurement error

SNPE and MCMC Performance on Krusell Smith with Winberry Solution Method

- Liu and Plagborg-Moller (2023) takes 6.5 min. per iteration
  - Likelihood evaluations compute intensive

- My approach that doesn't have to solve for a likelihood function

→ takes 30 seconds per iteration



Back

Empirical Application

Extending KS/Winberry to Full Information Empirical Application

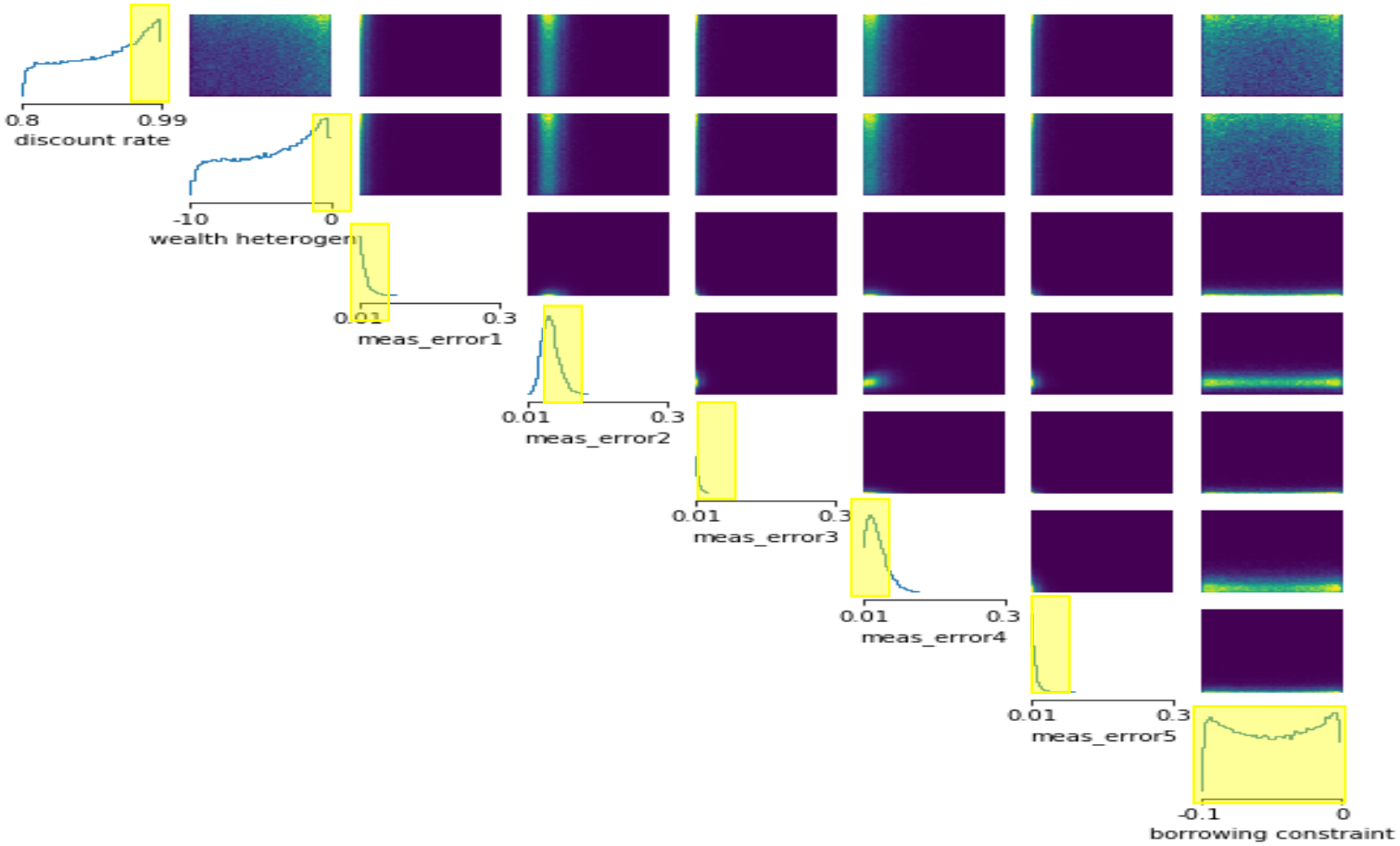
- KS/Winberry model estimated on empirical micro and macro data
  - Wealth data comes from Survey of Income and Program Participants (SIPP)
  - Macro data comes from FRED
  - Not attempted by Liu and Plagborg-Moller (2023)

- Difficulties: not all time periods are surveyed and…

- …each time step has different number of people surveyed

- Solution: break microdata down into percentiles and match distributions

Posteriors for Empirical Application



Conclusion

Conclusion

- I introduce the SNPE machine learning algorithm to improve the Bayesian estimation of structural macroeconomic models
- SNPE can also handle likelihood free estimation in both a Bayesian and MLE framework
- Empirical application estimating Krusell-Smith model solved via Winberry in a full information manner—with micro and macro moments
  - Extends Liu and Plagborg-Moller (2023)
- Code tutorial: https://shorturl.at/pMQ18

Thank You

camfen@umich.edu

https://cameronfen.github.io/

https://twitter.com/cameronfen1

Appendix

Contribution: More Accurate Posteriors than MCMC on Smets-Wouters

SNPE

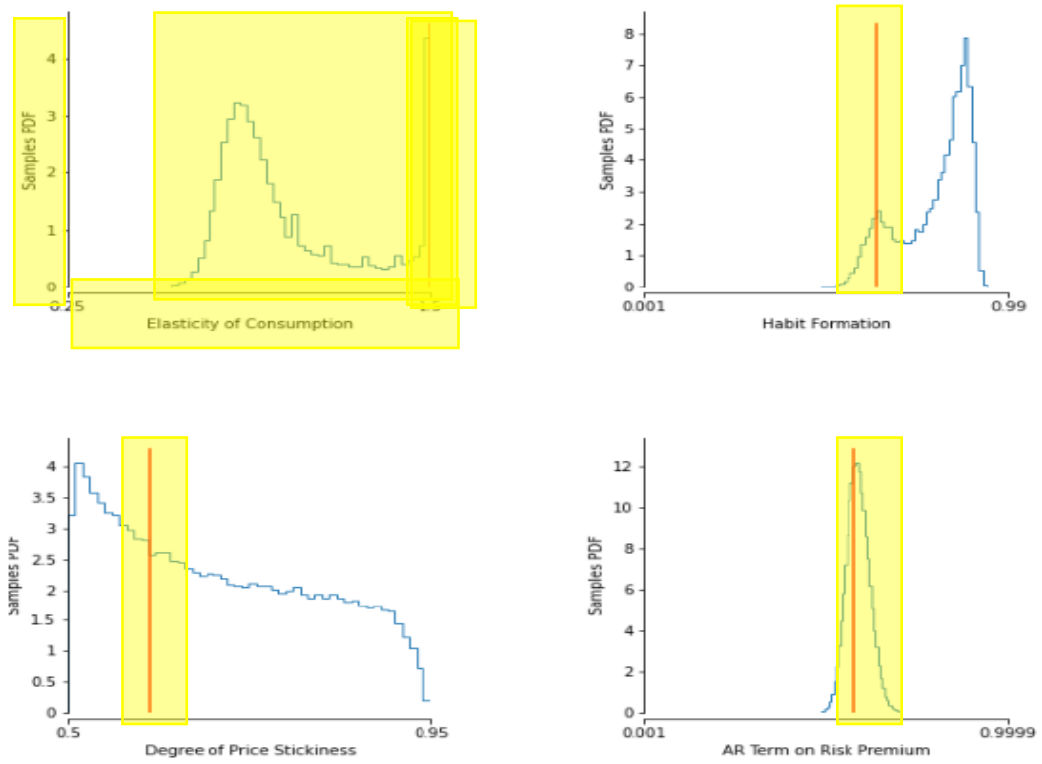


FIGURE I: EXAMPLE OF SNPE POSTERIORS

MCMC

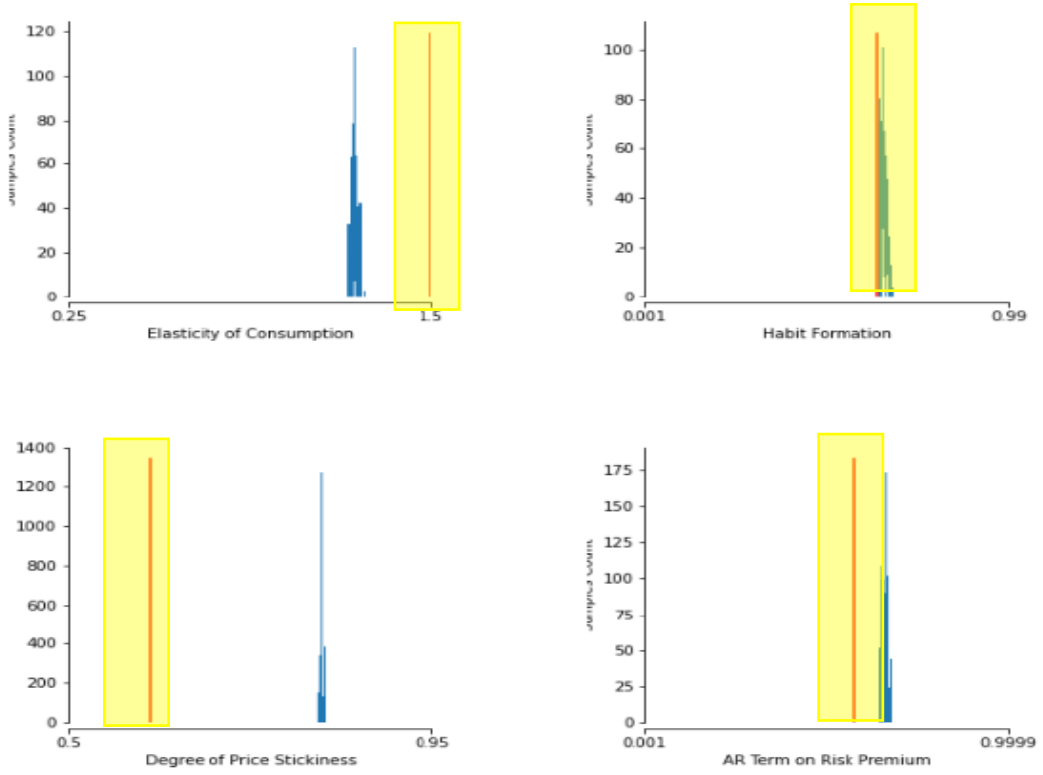


FIGURE II: EXAMPLE OF MCMC POSTERIORS

Additional Parameters Theoretical Rationale

Contribution Explained

- I introduce the SNPE machine learning algorithm to improve the Bayesian estimation of structural macroeconomic models
- SNPE can also handle likelihood free estimation in both a Bayesian and MLE framework
- Within my SNPE procedure, I also introduce normalizing flows, for high dimensional density estimation
- Empirical application estimating Krusell-Smith model solved via Winberry in a full information manner—with micro and macro moments
  - Difficult to do using current state-of-the-art likelihood methods

Literature

- Many models are non-smooth so one can't use perturbation and thus can't likelihood estimated without measurement error
  - **s-S models:** Arrow et. al. 1951, Caplin and Spulber 1987, Bertola and Caballero 1990, House and Leahy 2004, Caplin and Leahy 2006, Kahn and Thomas 2008, Caplin et. al. 2020, Badoczy 2022, etc.
  - **Structural Finance:** He, Whited, and Guo 2021, Terry, Whited, and Zakolyukina 2021, Taylor 2012, Whited and Guo 2006
- With non-smooth models: One generally must resort to simulation-based estimation:
  - **Method of Simulated Moments**: McFadden 1989, Pakes and Pollard 1989, Duffie and Singleton 1990, etc
  - **Machine Learning:** Kaji, Manresa, and Pouliot 2020

Back

Markov Assumption Versus State-Space Assumption

- With MSM, we impose the Markov Assumption on the data
  - A small number of lags captures all necessary historical information:

$$X_{t-20}, X_{t-19} \dots X_{t-2}, X_{t-1}, X_t \checkmark$$

Markov Assumption Versus State-Space Assumption

- With MSM, we impose the Markov Assumption on the data
  - A small number of lags captures all necessary historical information:

$$X_{t-20}, X_{t-19} \dots X_{t-2}, X_{t-1}, X_t$$

Markov Assumption Versus State-Space Assumption

- With MSM, we impose the Markov assumption on the data
  - A small number of lags captures all necessary historical information:

$$X_{t-20}, X_{t-19} \dots X_{t-2}, X_{t-1}, X_t$$

- With the state-space assumption, no information, no matter how back the data is, is not being conditioned, using the state of the model:

✓ ✓

$$X_{t-20}, X_{t-19} \dots X_{t-2}, X_{t-1}, X_t$$

Back

Why Do We Sample from the Joint?

- Bayes Rule:

$$P(\theta|X) = \frac{P(X|\theta)P(\theta)}{P(X)}$$

  - By assumption we cannot calculate the likelihood: $P(X|\theta)$
- Thus, we sample from the joint:

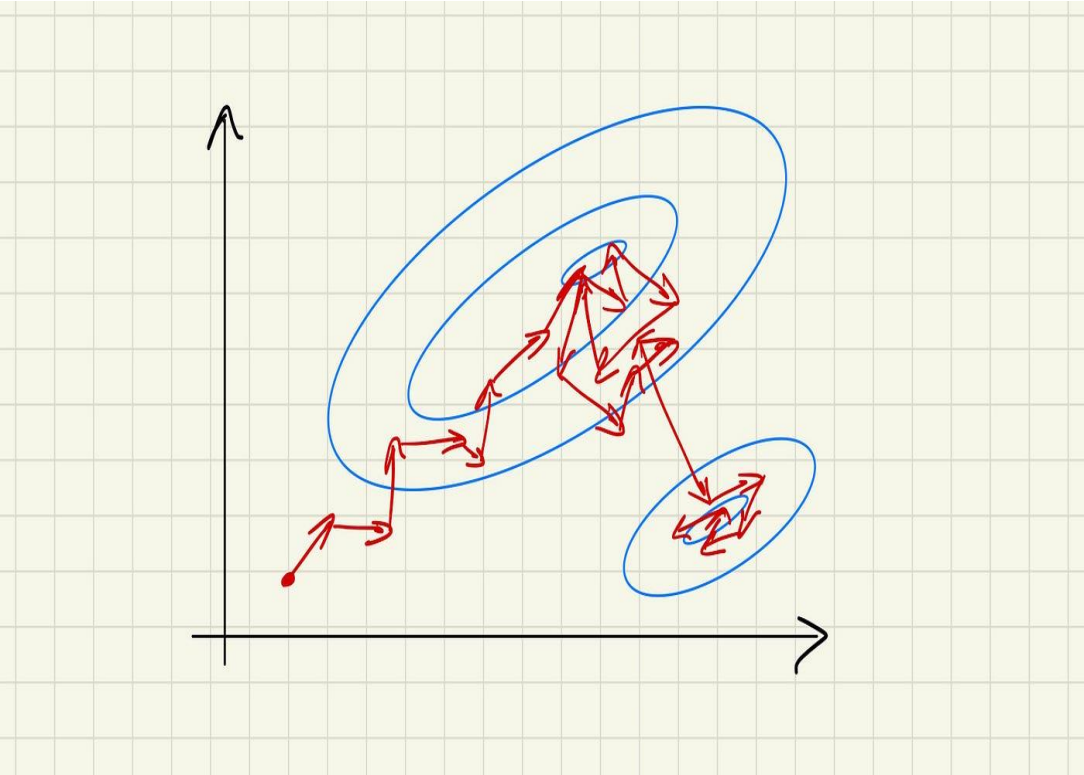
$$\{X_i^*, \theta_i^*\}_{i=1}^{I} \sim P(X|\theta)P(\theta) = P(X^*, \theta^*)$$

- Given samples from the joint, one could build a surrogate model, trained on these samples, to produce the unknown likelihood and then multiply by the prior to produce a posterior
- Other methods do this (SNLE) and is good intuition, however, SNPE skips a couple steps and builds a surrogate model to estimate the posterior directly

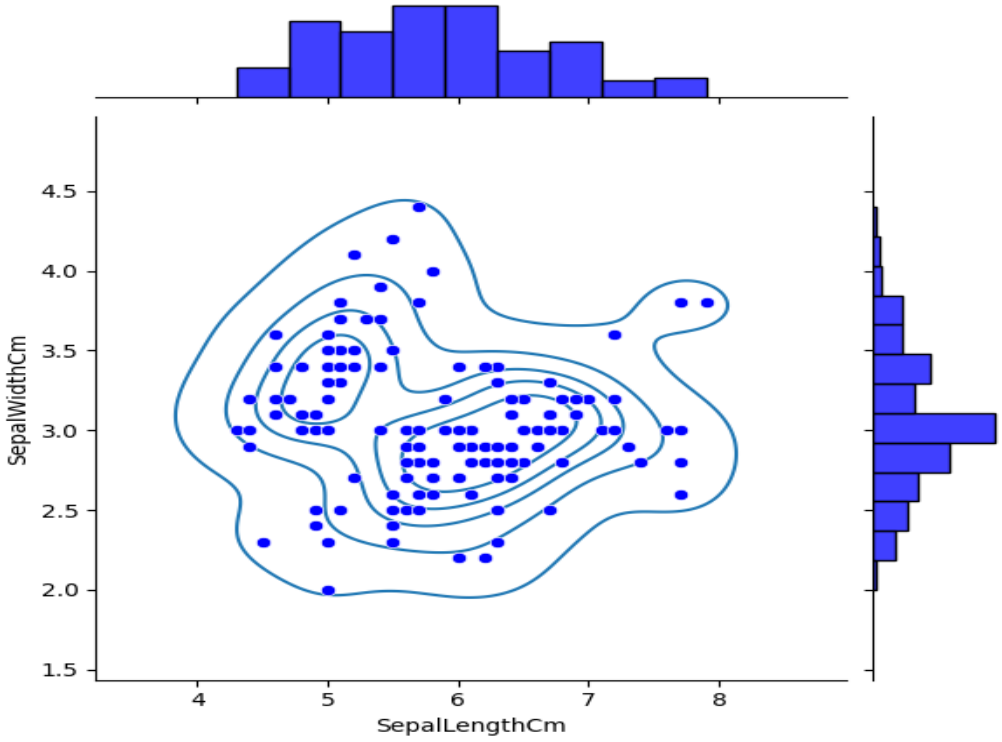
Back Bayes Back Step 3

High level differences: MCMC vs SNPE

MCMC is a Random Walk Algorithm



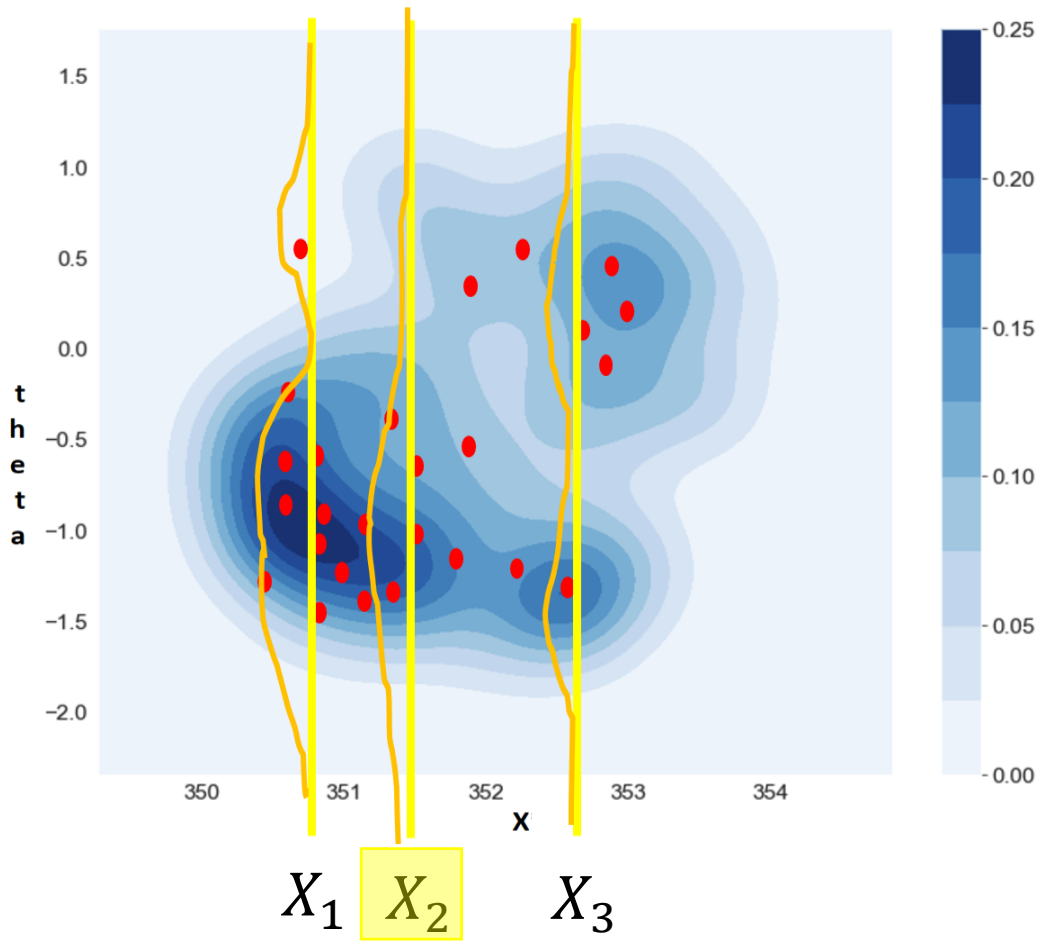
SNPE is a Posterior Density Estimator



Back

Step 2: Estimate the conditional density $p(\theta|X^*)$

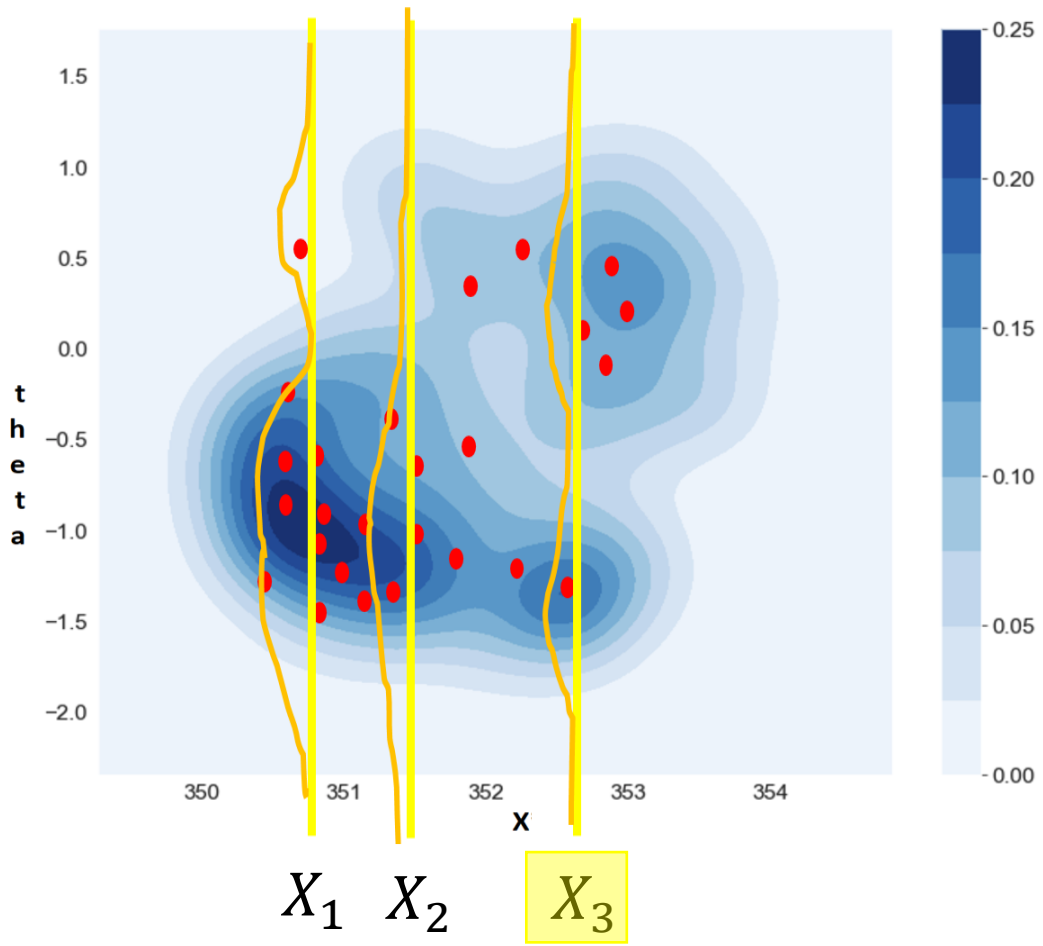
- High level: One way to estimate the conditional density is to build a parametric model that can evaluate $p(\theta|X_2)$ from joint samples and then maximize the conditional likelihood of p by updating its parameters
  - I will do this with a machine learning model: normalizing flows



Back

Step 2: Estimate the conditional density $p(\theta|X^*)$

- High level: One way to estimate the conditional density is to build a parametric model that can evaluate $p(\theta|X_3)$ from joint samples and then maximize the conditional likelihood of p by updating its parameters
  - I will do this with a machine learning model: normalizing flows



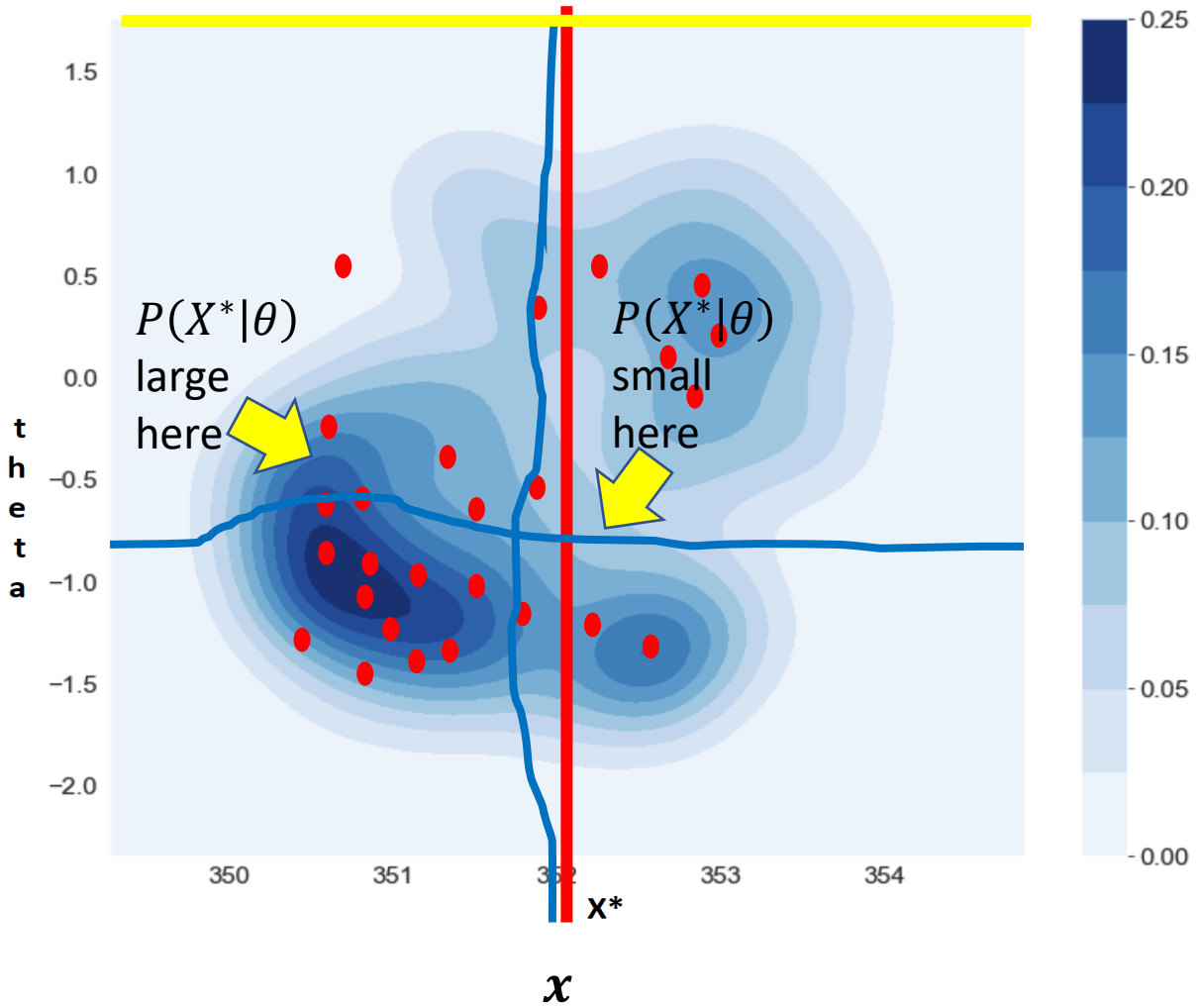
Back

Step 3.5: Multi-Round Inference

- When sampling from the prior $p(\theta)$, can sampling in places where the likelihood of the real data is small:

$$P(X^* = X|\theta) \text{ small}$$

- X^* samples far away from X → little information on posterior
- Solution: multi-round inference where the best guess of the posterior becomes the proposal distribution for sampling theta in the next round



Back

Talking Syntax

- Some syntax:
  - Call f_k a bijector (because it will be a bijection…more to follow)
  - A stack of f_k's is a flow
  - Subscript variables like the k in f_k and z_k correspond to the composition level
    - ie $\boldsymbol{\theta_K} = f_K \circ f_{K-1} \circ \cdots f_k \ldots f_1(\boldsymbol{\theta_0})$:
  - Superscript variables, $\boldsymbol{\theta_k^j}$ and f_k^j indicate the element in the vector $\boldsymbol{\theta_k}$ or f_k
    - $\boldsymbol{\theta_k}$ is a vector and f_k is a vector valued function
  - Next, will discuss the structure of individual f_k…
    - …with the understanding that we will compose many f_k's together to form a flow

Back

Flows: Preserving Invertibility

- The forward equation gives a way to find $\boldsymbol{\theta_{k+1}}$ from vector $\boldsymbol{\theta_k}$:

$$\theta_{k+1}^{0} = a_k^0 * \theta_k^0 + b_k^0$$

$$\theta_{k+1}^{j>0} = a_k^j(\theta_k^0, \theta_k^1 \dots \theta_k^{j-1}) * \theta_k^j + b_k^j\ (\theta_k^0, \theta_k^1 \dots \theta_k^{j-1})$$

- However, if one knows only knows vector $\boldsymbol{\theta_{k+1}}$, to find $\boldsymbol{\theta_k}$

$$\theta_k^0 = \frac{\theta_{k+1}^0 - b_k^0}{a_k^0}$$

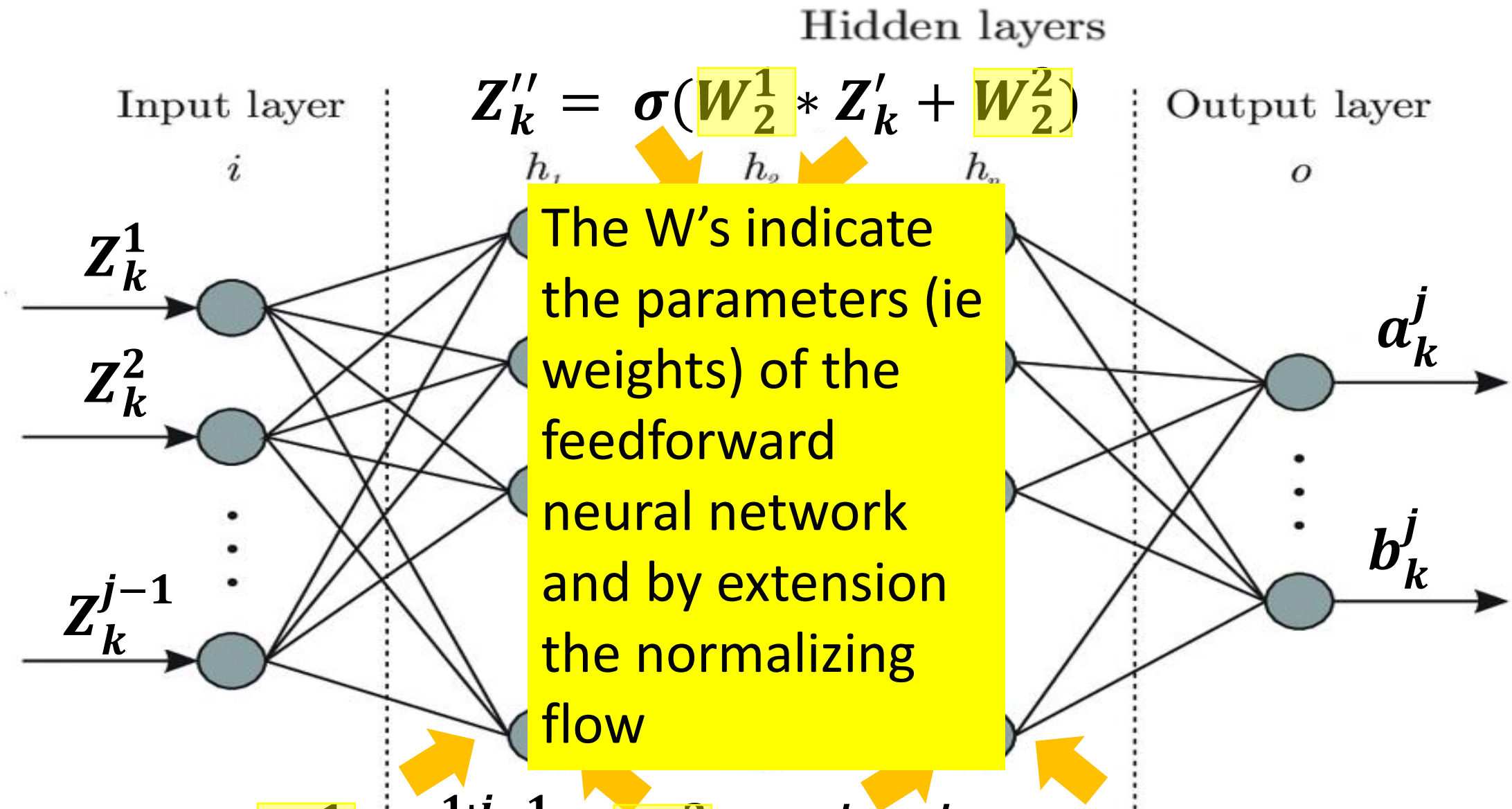
$$\theta_k^1 = \frac{\theta_{k+1}^1 - b_k^1(\theta_k^0)}{a_k^1(\theta_k^0)} \dots \text{induction for } j > 1$$

- Arbitrary conditioning: Permute θ_k^0 to θ_k^{J-1} before each bijector

Back

Feed-Forward Neural Networks

$$Z''_k = \sigma(W_2^1 * Z'_k + W_2^2)$$

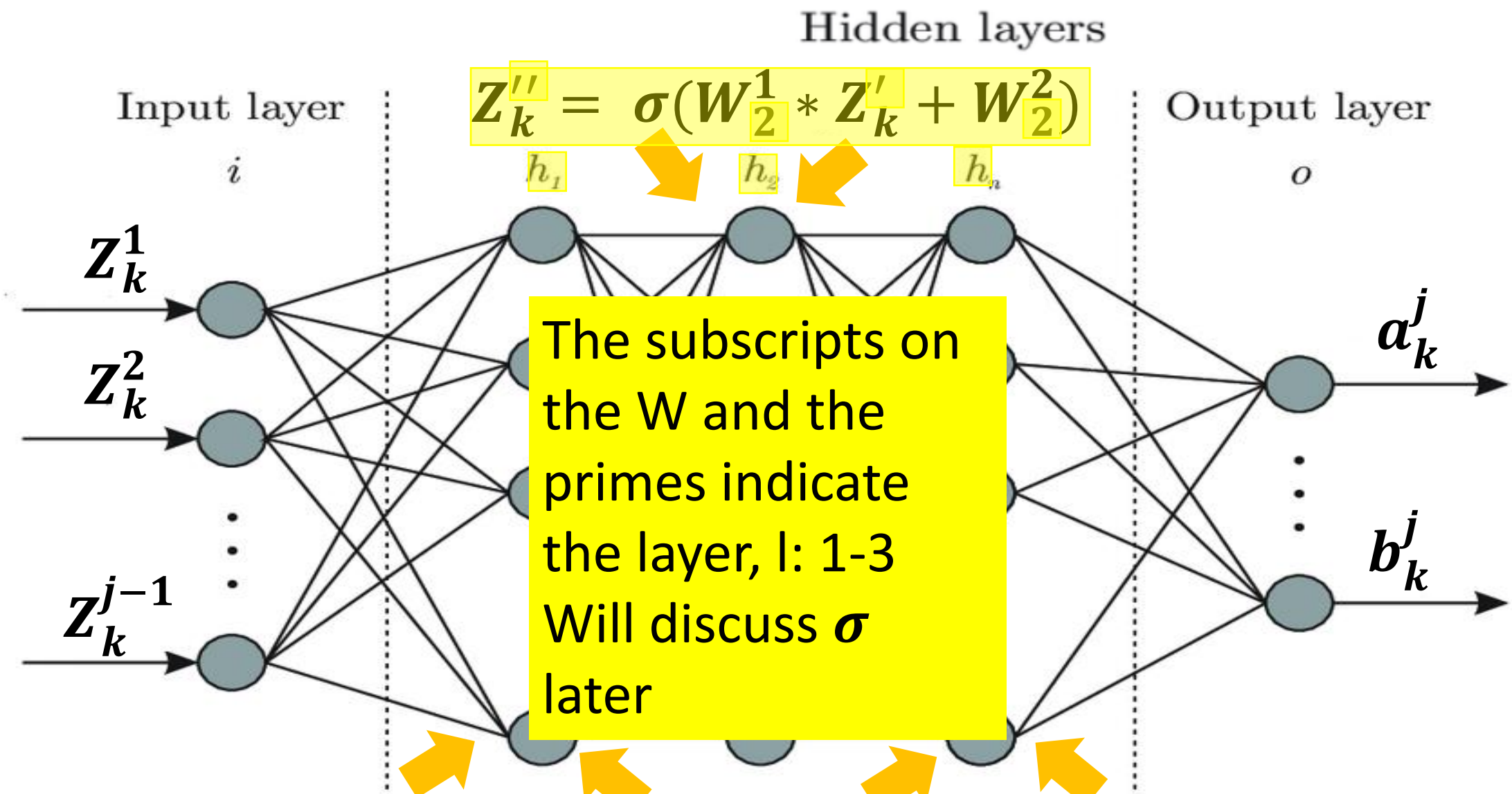


The W's indicate the parameters (ie weights) of the feedforward neural network and by extension the normalizing flow

$$Z'_k = \sigma(W_1^1 * Z_k^{1:j-1} + W_1^2)$$

$$[a_k^j, b_k^j] = W_3^1 * Z''_k + W_3^2$$

Feed-Forward Neural Networks



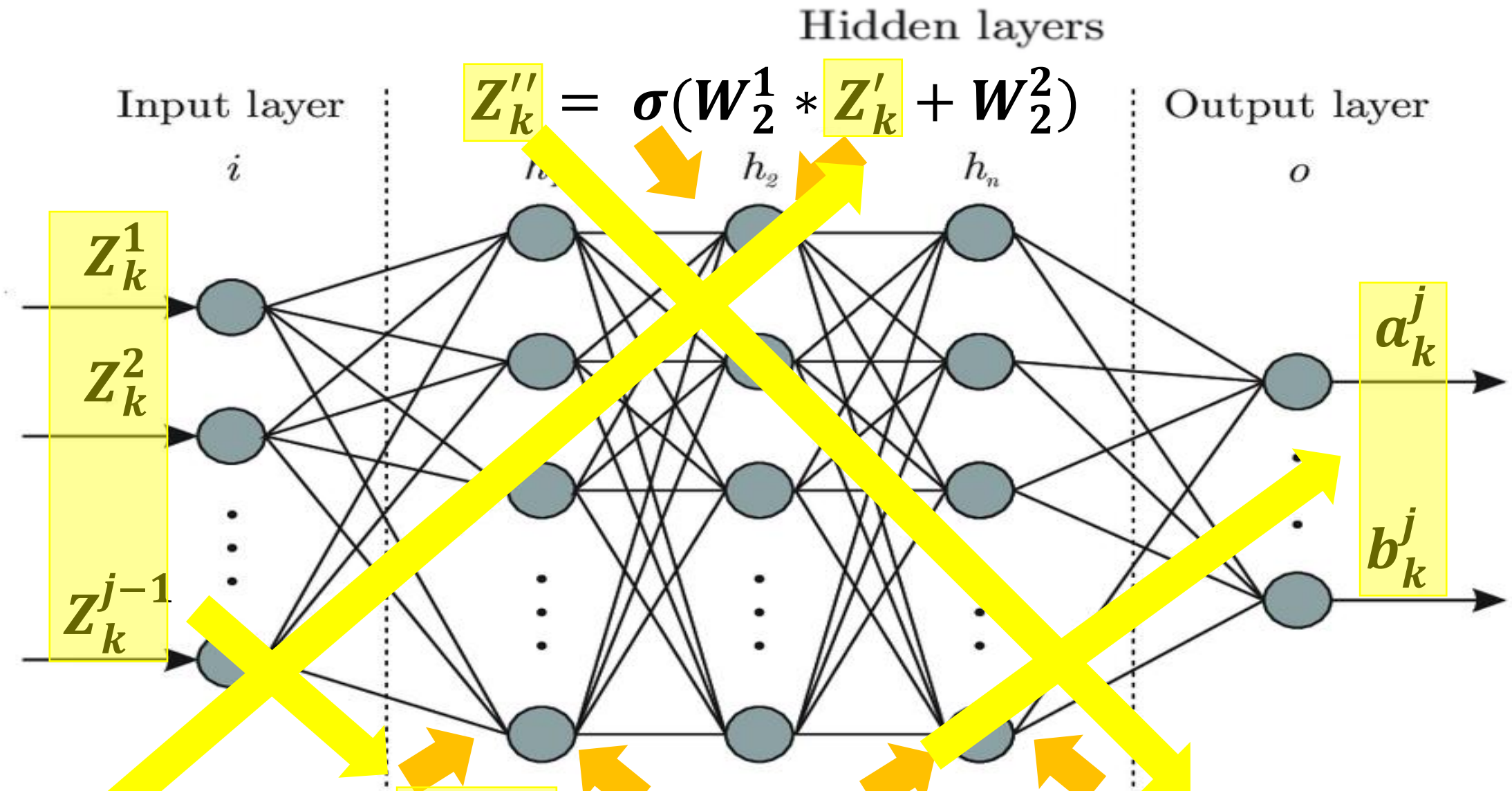
$$Z_k'' = \sigma(W_2^1 * Z_k' + W_2^2)$$

The subscripts on the W and the primes indicate the layer, l: 1-3
Will discuss σ later

$$Z_k' = \sigma(W_1^1 * Z_k^{1:j-1} + W_1^2)$$

$$[a_k^j, b_k^j] = W_3^1 * Z_k'' + W_3^2$$

Neural Networks: Composition of Linear Functions



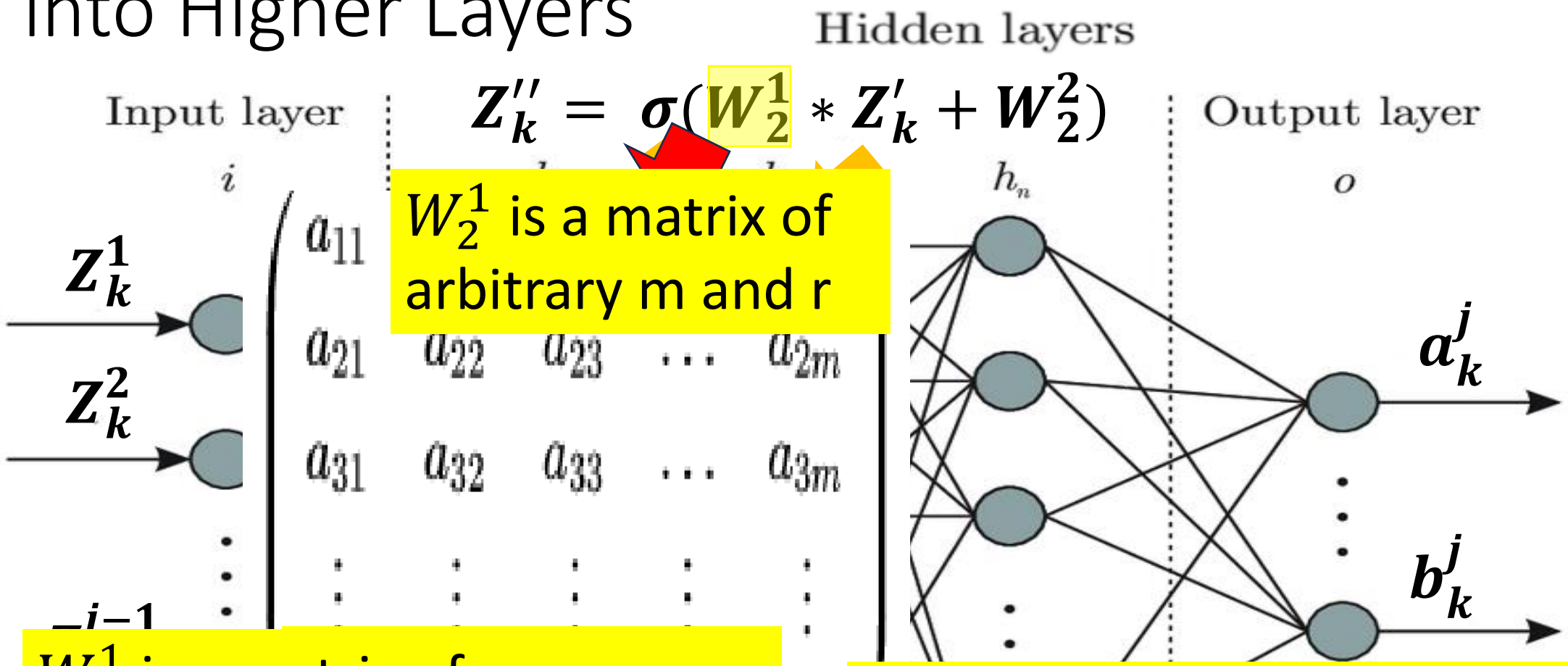
$$Z_k'' = \sigma(W_2^1 * Z_k' + W_2^2)$$

$$Z_k' = \sigma(W_1^1 * Z_k^{1:j-1} + W_1^2)$$

$$[a_k^j, b_k^j] = W_3^1 * Z_k'' + W_3^2$$

How Lower Layer Matrix Equations Feed into Higher Layers

$$Z''_k = \sigma(W_2^1 * Z'_k + W_2^2)$$



W_2^1 is a matrix of arbitrary m and r

W_1^1 is a matrix of arbitrary r by j-1

a vector j-1

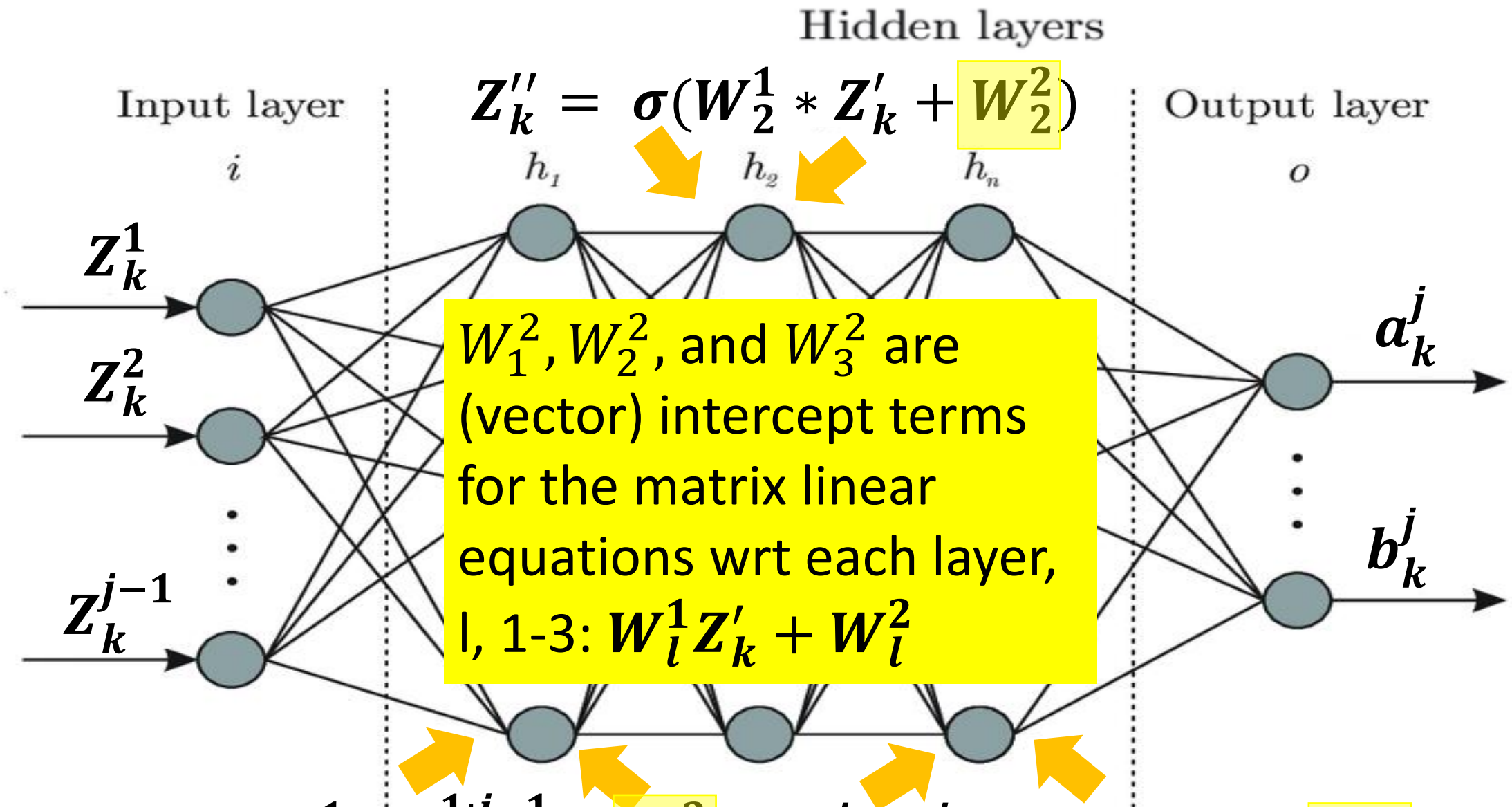
W_3^1 is a matrix 2 by m, as a_k^j, b_k^j are the two outputs

$$Z'_k = \sigma(W_1^1 * Z_k^{1:j-1} + W_1^2)$$

$$[a_k^j, b_k^j] = W_3^1 * Z''_k + W_3^2$$

Intercept Terms

$$Z''_k = \sigma(W_2^1 * Z'_k + W_2^2)$$

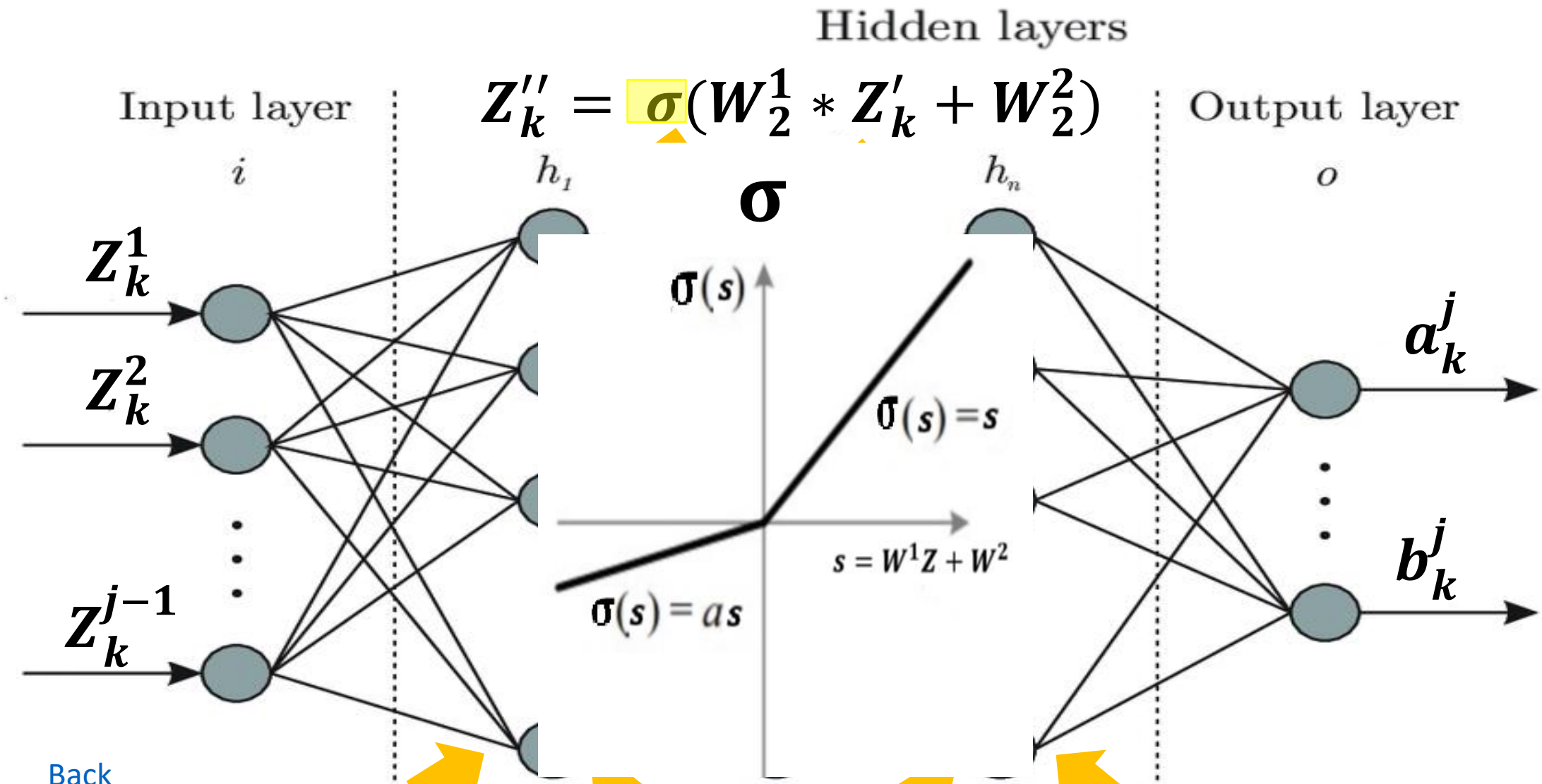


W_1^2, W_2^2, and W_3^2 are (vector) intercept terms for the matrix linear equations wrt each layer, l, 1-3: $W_l^1 Z'_k + W_l^2$

$$Z'_k = \sigma(W_1^1 * Z_k^{1:j-1} + W_1^2) \quad [a_k^j, b_k^j] = W_3^1 * Z''_k + W_3^2$$

What is σ?

$$Z''_k = \sigma(W_2^1 * Z'_k + W_2^2)$$



Back

$$Z'_k = \sigma(W_1^1 * Z_k^{1:j-1} + W_1^2) \quad [a_k^j, b_k^j] = W_3^1 * Z''_k + W_3^2$$

Constructing a Flexible Transformation f_k

- To be an invertible f_k with conditioning on $\boldsymbol{\theta_k}$, construct a function:

$$\theta_{k+1}^{0} = a_k^0 * \theta_k^0 + b_k^0$$

$$\theta_{k+1}^{j>0} = a_k^j(\theta_k^0, \theta_k^1 \dots \theta_k^{j-1}) * \theta_k^j + b_k^j(\theta_k^0, \theta_k^1 \dots \theta_k^{j-1})$$

  - Superscript, j, indicates the jth element of the θ_k array

$$\boldsymbol{\theta_K} = f_{K-1} \circ f_{K-2} \circ \cdots f_0(\boldsymbol{\theta_0})$$

  - Instead of being parameters, a_k^j and b_k^j are now trainable functions that take in $\theta_k^0, \theta_k^1 \dots \theta_k^{j-1}$

Syntax Invertibility Intuition

Constructing a Flexible Transformation f_k

- To be an invertible f_k with conditioning on $\boldsymbol{\theta_k}$, construct a function:

$$\theta_{k+1}^{0} = a_k^0 * \theta_k^0 + b_k^0$$

$$\theta_{k+1}^{j>0} = a_k^j(\theta_k^0, \theta_k^1 \dots \theta_k^{j-1}) * \theta_k^j + b_k^j(\theta_k^0, \theta_k^1 \dots \theta_k^{j-1})$$

  - Superscript, j, indicates the jth element of the θ_k array

$$\boldsymbol{\theta_K} = f_{K-1} \circ f_{K-2} \circ \cdots f_0(\boldsymbol{\theta_0})$$

  - Instead of being parameters, a_k^j and b_k^j are now trainable functions that take in $\theta_k^0, \theta_k^1 \dots \theta_k^{j-1}$

Syntax Invertibility Intuition

Flow Universal Approximator Intuition

- From Huang 2018
- First a sum of step functions → universal approximator of monotonic functions
- A logit can approximate a step function arbitrarily well → a sum of logit can do the same with monotonic functions
- A neural autoregressive flow can universally approximate any set of parameters and thus when using a logit nonlinearity, can approximate logits → step functions → monotonic functions arbitrarily well

Monotonic Functions and Universal Approximators

- One can map any uniform density to any distribution via the monotonic CDF
- One can map any distribution to the uniform density via the inverse CDF which also is monotonic
- Since you can map any distribution to a uniform via monotonic transformations and any uniform to any other distributions via monotonic transformation, then monotonic distributions can map any distribution to any other → universal approximation with flows

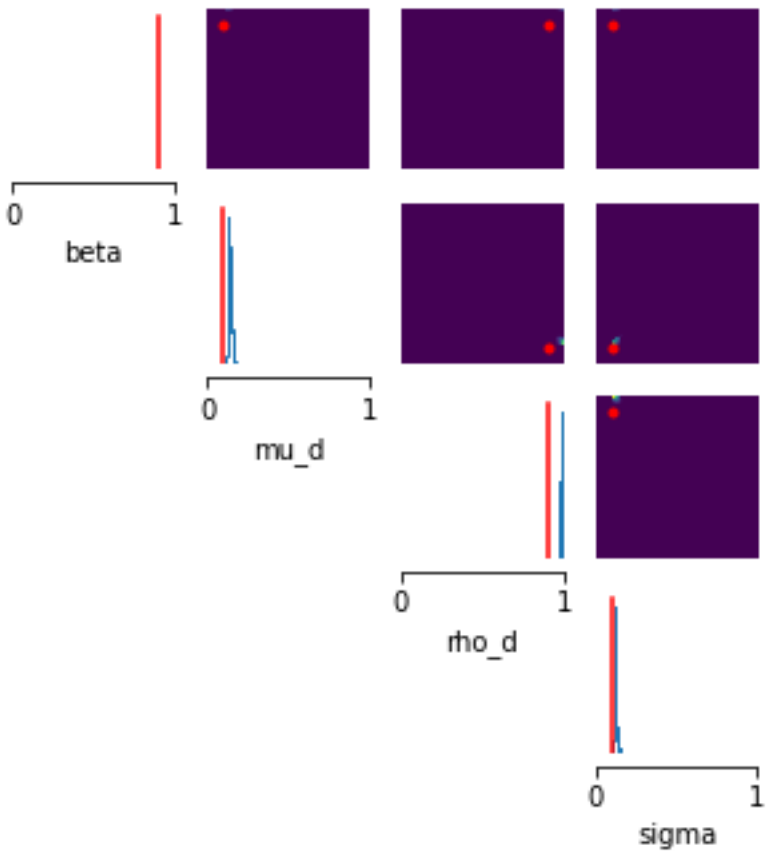
Back

Lossless Embedding

- I pointed out that in comparison to (Ahn et. al. 2018), the use of an end to end embedding generally leads to lossless compression
- Since a small number of parameters generate many sample data points, there likely exists a sufficient statistic the size of the parameter set
  - Modulo continuity and other regularity conditions
- By Rao-Blackwell, if there exists a sufficient statistic, it will maximize likelihood
  - Caveat regarding global versus local mins
- A universal approximator embedding neural network, can arbitrarily approximate any function so given enough parameters can approximate the sufficient statistic and will approximate the sufficient statistic as that will minimize MLE loss
- Using PCA like in Ahn et.al. (2018) will not be lossless as it’s not trained end to end with the Bayesian objective

Projection Model

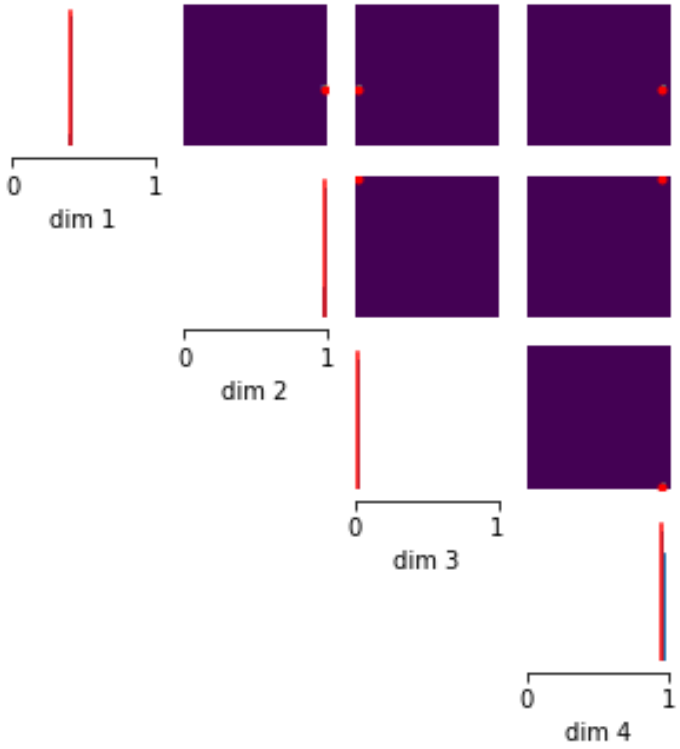
- Lucas-Tree estimated via projection
- Can see some errors with the posterior not quite aligned with the data generating parameters
- No measurement error used when estimating



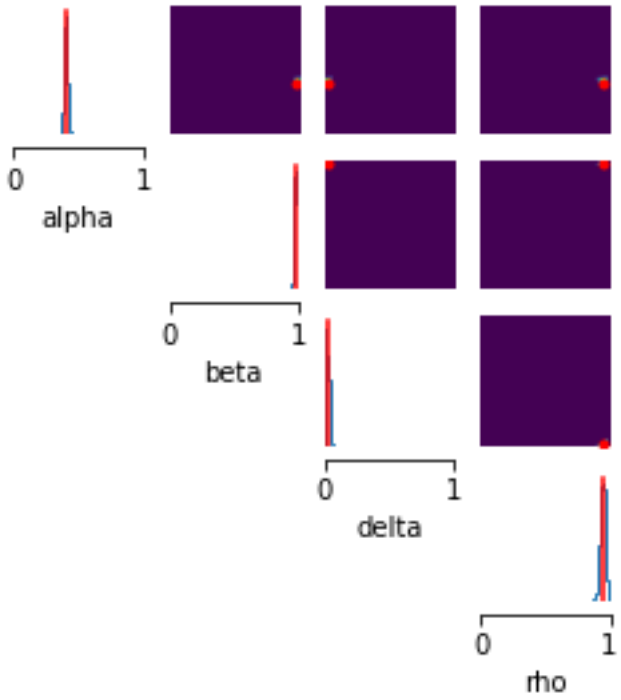
Back

RBC Model Estimated Two Ways

MCMC Ground Truth Posterior

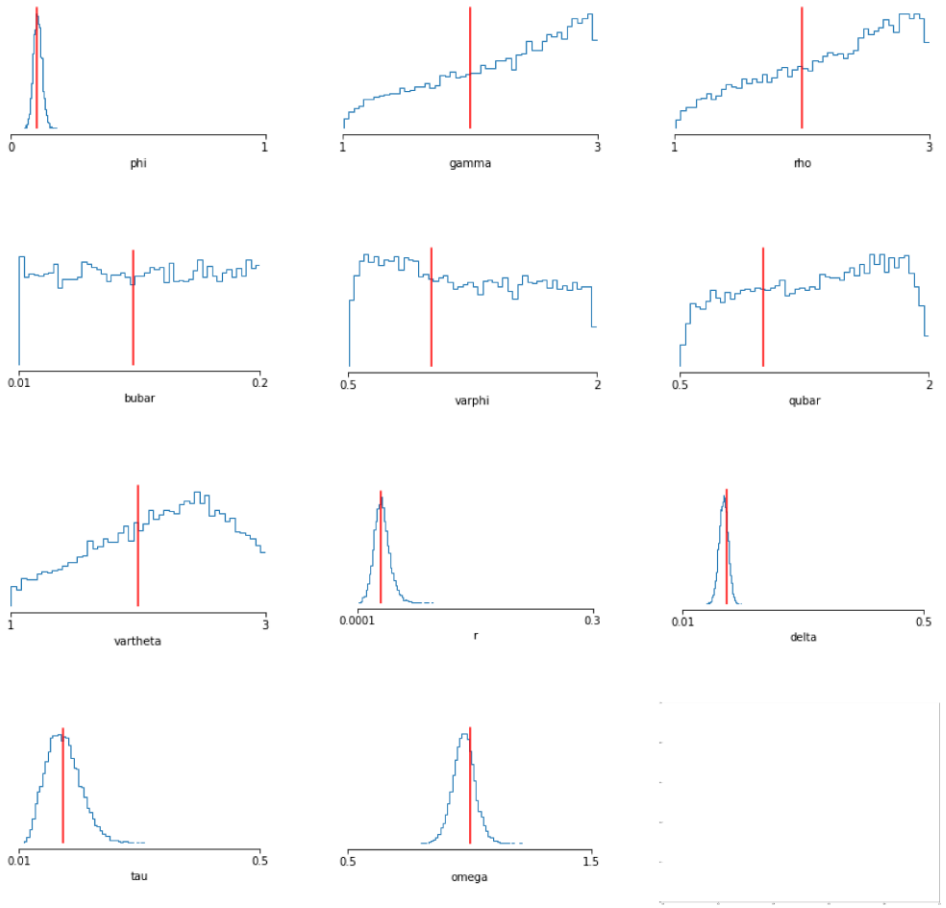


SNPE Posterior



[Back]

Bequests Value Function Iteration Model



- Liquid and Illiquid asset
  - Kink at the transition point → No perturbation
- Solved via value function iteration
- Using measurement error

[Back]

SNPE

Algorithm 1: SNPE Algorithm

Input: Simulator $p(X^*|\theta)$, prior $p(\theta)$, data X, Density Estimator $p_\phi(X^*|\theta)$, Rounds R, Samples S;

Initialize: Posterior $p^{(0)(\theta)} = p(\theta)$, data set D $= \{\}$;

for $i \leftarrow 1$ **to** R **do**

  Sample $\theta^{*(n)} \sim p^{(i-1)}$ for $n = 1...S$ with Monte Carlo;

  Simulate $X^{*(n)} \sim P(X^*|\theta^{*(n)})$ for $n = 1...S$;

  Concatenate data $D = D \cup \{X^{*(n)}, \theta^{*(n)}\}_{n=1}^{S}$;

  while $p_\phi(\theta|X^*)$ *not converged* **do**

    Sample $\{X^{*(i)}, \theta^{*(i)}\}_i^B \sim D$ from D;

    Train $p_\phi(\theta|X^*)\dfrac{p^{(i-1)}(\theta)}{p(\theta)}$ on $\{X^{*(i)}, \theta^{*(i)}\}_i^B$;

  end

  Update posterior $p^{(i)(\theta)} = p_\phi(\theta|X^* = X)$;

end

Back

Alternative Algorithm: Sequential Neural Variational Inference

Algorithm 3: SNVI Algorithm

Input: Simulator $p(x|\theta)$, prior $p(\theta)$, data x_0, discriminator $d_\phi(x,\theta)$, Rounds R, Samples S;

Initialize: Posterior $p^{(0)} = p(\theta)$, data set D = {};

for $i \leftarrow 1$ **to** R **do**

  Sample $\theta^{(n)} \sim f_{\phi'}^{(i-1)}(\theta)$ for $n = 1 \ldots S$;

  Simulate $x^{(n)} \sim P(x|\theta^{(n)})$ for $n = 1 \ldots S$;

  Concatenate data $D = D \cup \{x^{(n)}, \theta^{(n)}\}_{n=1}^{S}$;

  while $d_\phi(x,\theta)$ *not converged* **do**

    Sample $\{x^{(i)}, \theta^{(i)}\}_i^B \sim D$ from D;

    Sample K contrasting samples for each i sample $\{x'^{(k)}\}_k^{B*K}, \{\theta'^{(k)}\}_k^{B*K} \sim D\{x\}, D\{\theta\}$, breaking the joint distribution of D into independent marginals;

    Train the GAN using the GAN loss function from the two distributions x, θ and x', θ' and a softmax/multinomial logit objective $\sum_i^B log(\frac{exp(d_\phi(x^{(i)}, \theta^{(i)})}{\sum_k^K exp(d_\phi(x^{(k)}, \theta^{(k)})})$;

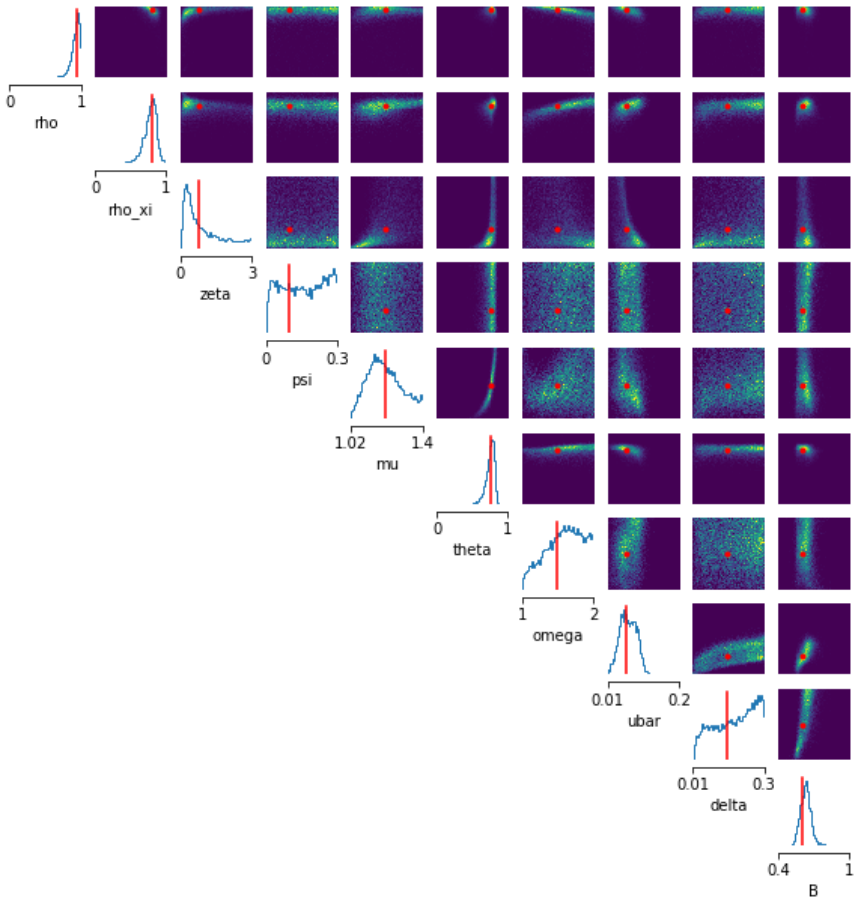
  end

  Train normalizing flow with variational inference $f_{\phi'}^i = argmin_{\phi'}(KL(f_{\phi'}^i(\theta)||exp(d_\phi(x,\theta))p(\theta))$;

end

Back

HANK Model Solved via Reiter



- Using Reiter's method, this HANK model has 1000+ states
- Easy to simulate from, but takes 20-30 mins to use the Kalman filter for a single likelihood evaluation, which is intractable
- Other option is to use dimensionality reductions which loses information (Ahn et. al. 2018)

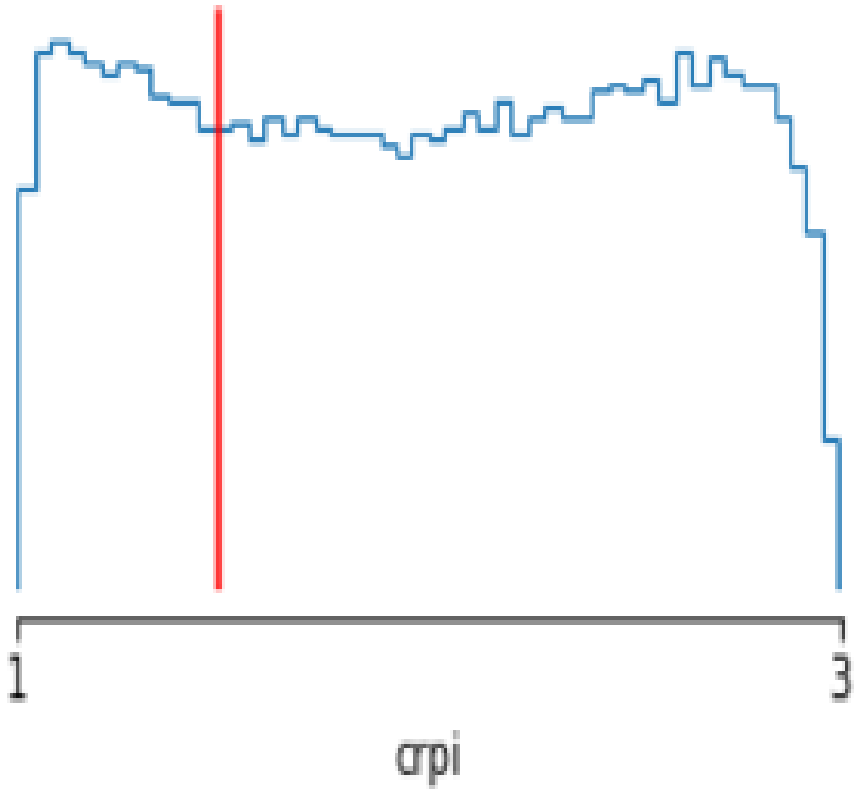
Back Back LPM

Discussion on Multimodality

Multimodality and Frequentism

- Weak identification and multimodality is a problem for frequentist statistics → what is the right parameter value…
- …but is just another posterior in the Bayesian approach
- Furthermore, multimodality might provide important feedback to economic research

Taylor Rule Parameter on Inflation



- Crpi, the Taylor rule parameter on inflation is unidentified
- SW and current data cannot distinguish between the two stories of central bank inflation fighting ability

Multimodality and economic cycles

- Like the Diamond Diybvig (1983) model of bank runs, it doesn't require much imagination to think we coordinate on multiple equilibria

- In good times we coordinate on parameter values that lead to high output, but perhaps there is a low consumption, low production equilibrium that we could coordinate on if we get knocked out of the good equilibrium

- A deeper understanding of equilibrium coordination, by its nature involves deeper understanding of the multiple modes of parameter posteriors